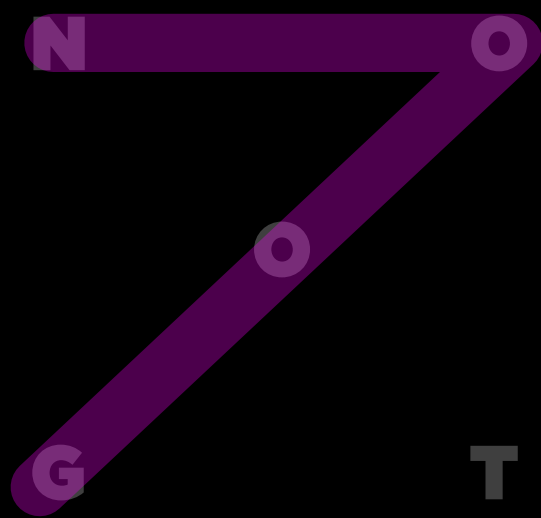
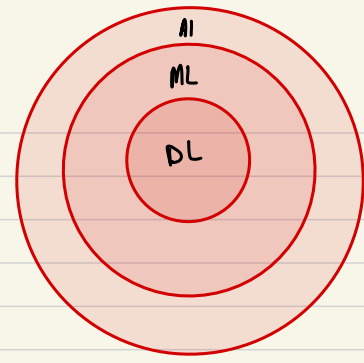


AIS



Made with :->
Claude & Gemini

What is AI?



- A branch of computer science to create intelligent machines that can behave, think and make decisions like humans
 - Allows machines to work with own intelligence without being pre-programmed for specific tasks
 - Combines technologies like machine learning, reasoning, problem-solving, perception and natural language processing

Importance of AI:

- Solve real-world problems accurately (health, marketing, traffic etc.)
- Create virtual personal assistants (Siri, Alexa etc.)
- Build robots for high-risk environments
- Enable new technologies and opportunities

Goals of AI:

- Replicate human intelligence.
- Solve knowledge-intensive tasks.
- Intelligently connect perception and action.
- Perform tasks requiring human intelligence (e.g., proving theorems, playing chess, surgical planning, driving in traffic).
 - Exhibit intelligent behavior, learn autonomously, demonstrate, explain, and advise.

Advantages of AI:

- High accuracy with fewer errors.
- High-speed decision-making.
- High reliability in performing tasks.
- Useful in risky situations.
- Provides digital assistance.
- Enhances public utilities like self-driving cars, facial recognition, natural language processing.

Disadvantages of AI:

- High cost of hardware and software.
- Limited ability to think creatively or outside programmed parameters.
- Lacks emotions and feelings, leading to potential harm if not carefully managed.
 - Increases dependency on machines, potentially diminishing human mental capabilities.
- Lacks original creativity compared to human intelligence.

AI Problems:

Artificial Intelligence (AI) aims to create computer systems that exhibit intelligent behavior similar to human intelligence. However, intelligence itself has inherent limitations in areas like perception, memory, and computational capabilities. AI research seeks to understand the underlying computations required for intelligent behavior across various aspects:

- Perception (vision, speech recognition)
- Communication through natural languages
- Reasoning and commonsense understanding
- Planning and decision-making
- Learning from data and experiences
- Memory and knowledge representation

As AI progressed, some fundamental questions arose that need addressing:

- 1) What are the core assumptions about the nature of intelligence and the cognitive processes that define it?
- 2) What techniques will prove effective in solving the diverse array of AI problems across different domains? Is there a common set of techniques applicable?
- 3) At what level of detail should AI systems model or replicate human-level intelligence? Is achieving functional parity the goal?
- 4) How can we definitively evaluate when an AI system has attained true intelligence? What are the benchmarks for success?

The early work focused on structured, formal tasks like:

- Game playing (chess, checkers)
- Theorem proving in mathematics and logic

The initial assumption was computers could excel by brute-force, exploring all solutions leveraging raw speed. However, it was realized these problems require significant knowledge representation.

As research advanced, AI started tackling more diverse areas mapped to different task domains:

Mundane Tasks:

- Simple, routine tasks that can be automated

Formal Tasks:

- Games, mathematical proofs, program verification

Expert Tasks:

- Engineering design, scientific analysis
- Medical diagnosis, financial planning

In summary, AI addresses problems across mundane, formal and expert task domains requiring techniques for perception, language, reasoning and learning.

AI techniques:

AI techniques aim to represent knowledge in a generalized, human-understandable way that is modifiable and can handle incomplete/inaccurate knowledge.

Intelligence requires knowledge, which has various challenging properties.

- Huge volume
- Difficult to characterize accurately
- Constantly changing
- Organized differently from data
- Complex nature

To address these challenges, AI techniques incorporate several key approaches:

1 Search Techniques:

- AI systems often need to explore large solution spaces to find optimal or satisfactory solutions.
- Search algorithms like depth-first, breadth-first, and heuristic search systematically navigate through these spaces.
- They leverage strategies like pruning, backtracking, and informed search (using heuristics) to improve efficiency.

2 Knowledge Representation:

- Knowledge must be represented in a form that is understandable to both humans and machines.
- Techniques like rules, frames, semantic networks, and logic facilitate this representation.
- They capture generalizations, group similar concepts, and enable reasoning and inference.

3 Abstraction:

- Abstraction is the process of identifying and representing essential features while ignoring irrelevant details.
- It helps manage complexity by focusing on high-level concepts and patterns.
- Techniques like hierarchical abstraction, problem decomposition, and constraint propagation are employed.

By combining search techniques, knowledge representation and abstraction methods, AI systems can effectively represent, reason with, and adapt knowledge to exhibit intelligent behavior in various domains.

Example of AI Technique:- Tic Tac Toe

Approach 1: Brute-Force with Lookup Table (MOVETABLE)

- Algorithm:
 - 1 Represent the current board state as a vector.
 - 2 Convert the vector to a decimal number.
 - 3 Use the decimal number as an index to look up the optimal move in a massive pre-computed table called "MOVETABLE". This table contains entries for every possible board configuration.
 - 4 Update the board state based on the recommended move from the table.
- Comments:
 - This approach is fast but incredibly inefficient in terms of space. The "MOVETABLE" requires immense storage space to hold optimal moves for all possible board configurations.
 - Creating and maintaining such a table is labor-intensive and prone to errors.
 - It lacks generalizability - this technique wouldn't scale well to larger or more complex games due to the exponential explosion in possible board configurations.

Approach 2: Strategic Programming with Subroutines

- Data Structures:
 - Board: A nine-element vector representing the board state (0: blank, 1: X, 2: O).
 - Turn: An integer indicating the current move number (1-9).
- Algorithm:
 - 1 Utilize three subroutines:
 - Make2: Returns the center square if blank, otherwise, a random non-corner blank square.
 - Posswin(p): Checks rows/columns/diagonals, returns 0 if player p can't win next move, else returns the winning square number
 - Go(n): Makes a move to square n, updating the board and turn.
 - 2 Implement a pre-programmed strategy based on the current turn (e.g., prioritize center or corner squares for X, prioritize blocking opponent's win for O).

- Comments:
 - This approach is more space-efficient than the first one and easier to understand/modify due to the explicit strategy.
 - However, the strategy is still programmed manually, limiting its ability to generalize to other games or situations.

Approach 3: Lookahead and Evaluation (More Advanced)

- Algorithm:
 - 1 Evaluate all possible next moves and the opponent's potential replies.
 - 2 For each move, simulate the game forward for several steps, estimating the likelihood of winning from each possible future state.
 - 3 Choose the move that leads to the most promising future outcome for the AI player.
- Comments:
 - This approach offers better decision-making compared to the previous two but requires significant computational resources and sophisticated algorithms.
 - Requires knowledge of overall game strategy beyond basic tactics.

Levels of AI Models:

- Level 1: Programs that solve trivial problems easily, that are solvable by computers without AI techniques
 - Examples: EPAM (memorizing syllables) - It is of interest only to psychologists
- Level 2: Programs that solve non-trivial problems using AI techniques, with the goal of modeling human performance
 - Reasons for modeling human performance:
 - Testing psychological theories of human performance (e.g., PARRY - simulating paranoid behavior)
 - Enabling computers to understand human reasoning (e.g., answering questions based on human behavior)
 - Allowing humans to understand computer reasoning (transparency and acceptance)
 - Exploiting knowledge from experts (e.g., GPS - General Problem Solver)

Key Characteristics:

- Level 1 programs use direct methods without AI techniques
- Level 2 programs use AI techniques, including:
 - Search (when no direct method is available)
 - Knowledge representation (about problem domain objects)
 - Abstraction (for pruning and real-time solutions)

Modeling Human Performance:

- Two approaches:
 - Producing programs that solve problems the same way humans do
 - Producing programs that solve problems in the easiest way possible

In summary, AI models can either solve trivial problems directly or tackle non-trivial problems using AI techniques like search, knowledge representation, and abstraction. The goal for the latter is to model human performance, either by replicating human reasoning or finding the most efficient solution, while accounting for factors like transparency, acceptance, and expert knowledge.

Criteria for success:

Defining success criteria is critical for AI projects. A key question is "How do we determine if a machine is truly intelligent?" Turing proposed the "Turing Test" in 1950 to evaluate this, involving an interrogator interacting with a human and machine to determine which is which. However, concerns exist about the vast knowledge required for machines to pass.

As an alternative, measuring achievement in restricted domains is explored. In areas like chess, programs can earn ratings based on defeating human players. For tasks like chemical analysis, qualitative assessments compare program outputs to published research.

When aiming to simulate human performance, success is measured by how closely a program's behavior matches humans', using techniques from psychology. The goal is not outperforming humans on all tasks, but replicating their problem-solving approaches, including failures.

Ultimately, determining true machine intelligence remains challenging. However, constructing programs that meet defined performance standards within specific domains is achievable.

For AI programs, specifying criteria for success within their intended restricted domains is crucial, as this currently provides the best measure of progress in the field.

Problem, Problem space and Search:

To build a system capable of solving a particular problem, four key steps are required:

- Defining the Problem: Precise definition including initial and final situations.
- Problem Analysis: Identifying crucial features for selecting appropriate problem-solving techniques.
- Task Knowledge Representation: Isolating and representing essential task knowledge.
- Selecting Problem-Solving Techniques: Choosing the most suitable techniques for the problem at hand.

State space search is a common approach in Artificial Intelligence, particularly for games. It represents a problem as a series of states and actions (operators) that transition between them. Key components of a state space include:

- States: All possible configurations of the problem (e.g., board positions in a game).
- Operators: Actions that change the state (e.g., legal moves in a game).
- Initial State: The starting point of the problem.
- Goal States: The desired end states (often identified by a program).

Defining the chess problem as state space search:

Formalizing Chess as a Problem:

- Define the initial state (starting chessboard position).
- Define the legal moves as operators that change the board state.
- Define the goal state as a position where the opponent's king is under attack with no legal moves remaining.

Challenges of Defining Legal Moves with Explicit Rules:

- Enormous number of rules needed (around 10^{120} for all possible board positions).
- Difficulty in storing and handling such a massive rule set.

Solution: Representing Moves with Patterns and Substitutions:

- Develop a general way to describe legal moves using patterns and substitutions.
- This reduces the number of rules needed and improves program efficiency.

Chess as a State Space Search:

- The chessboard represents the current state (one of many possible states).
- Legal moves act as operators, transitioning the board from one state to another.
- AI systems search this state space to find a path from the initial state to a winning goal state.

Defining the jug problem as state space search:

You have two jugs, one with a 4-gallon capacity and another with a 3-gallon capacity. Neither jug has any measuring markers. There is a pump to fill the jugs with water. The task is to get exactly 2 gallons of water into the 4-gallon jug.

Defining the Problem as a State Space:

- The state space is represented by ordered pairs of integers (x, y) , where:
- x represents the number of gallons in the 4-gallon jug (0, 1, 2, 3, or 4)
- y represents the number of gallons in the 3-gallon jug (0, 1, 2, or 3)
- The start state is $(0, 0)$, indicating both jugs are empty.
- The goal state is $(2, n)$, where n can be any value, as the problem only specifies getting 2 gallons in the 4-gallon jug.

↳ Production Rules $\Rightarrow$

$$\left[\begin{array}{l} (x, y) \rightarrow (4, y) \\ \text{If } x > 4 \end{array} \right.$$

$$\left[\begin{array}{l} (x, y) \rightarrow (x, 3) \\ \text{If } y > 3 \end{array} \right.$$

$$\left[\begin{array}{l} (x, y) \rightarrow (x-1, y) \\ \text{If } x > 0 \end{array} \right.$$

$$\left[\begin{array}{l} (x, y) \rightarrow (x, y-1) \\ \text{If } y > 0 \end{array} \right.$$

$$\left[\begin{array}{l} (x, y) \rightarrow (0, y) \\ \text{If } x > 0 \end{array} \right.$$

$$\left[\begin{array}{l} (x, y) \rightarrow (x, 0) \\ \text{If } y > 0 \end{array} \right.$$

$$(x, y) \rightarrow (4, y - (4 - x))$$

If $x + y \geq 4$ and $y > 0$

$$(x, y) \rightarrow (x - (3 - y), 3)$$

If $x + y \geq 3$ and $x > 0$

$$(x, y) \rightarrow (x + y, 0)$$

If $x + y \leq 4$ and $y > 0$

$$(x, y) \rightarrow (0, x + y)$$

If $x + y \leq 3$ and $x > 0$

$$(0, 2) \rightarrow (2, 0)$$

$$(2, y) \rightarrow (0, y)$$

Diagram for memorising $\Rightarrow$

Checking which one
is the 4 gallon one $\leftarrow$
and which one is
3 gallon $\leftarrow$

0, 0
4, 3
=, =
0, 0

0, 3

3, 3

4, 2

3, 3

4, 2

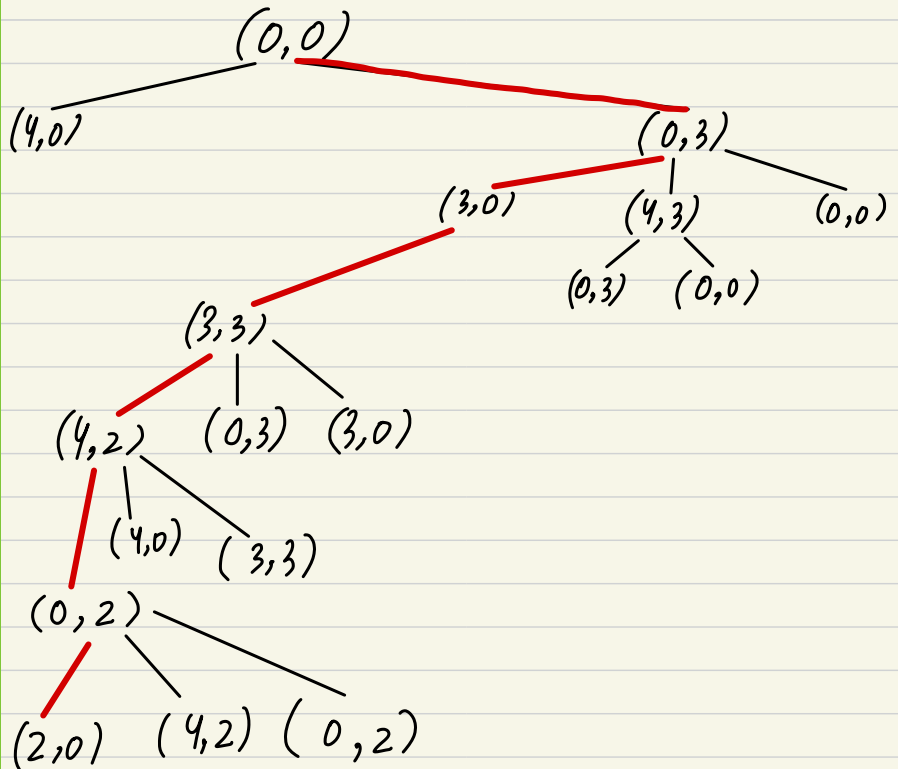
0, 2

2, 0

Same water level $\leftarrow$

Goal Achieved $\leftarrow$

State space tree $\Rightarrow$



Production Systems:

Production systems are cognitive architectures used in artificial intelligence to implement search algorithms and emulate human problem-solving skills. They consist of rules and actions encoded to solve specific problems efficiently.

Elements of a Production System:

- 1) Global Database: This is the primary database containing all the necessary information to complete a task. It is divided into temporary and permanent parts.
- 2) Production Rule Set: A set of rules that operate on the global database, each consisting of a precondition and a postcondition.
- 3) Control System: The decision-maker that determines which production rule to apply and when to stop the computation based on termination conditions.

Working Principle:

The production rules operate on the global knowledge database. Each rule has a precondition that is either satisfied or not by the current state of the database. If the precondition is satisfied, the rule can be applied, and its corresponding action is executed, changing the global database. The control system determines which applicable rule should be fired and stops the computation when a termination condition is met.

Classes of Production Systems:

- 1) Monotonic Production System: Allows simultaneous application of rules without interference.
- 2) Non-monotonic Production System: Efficiently solves problems without backtracking.
- 3) Commutative System: Order of operations is not critical, suitable for reversible changes.
- 4) Partially Commutative Production System: Results can be achieved by interchanging the states of rules.

Features of Production Systems:

Simplicity: The use of the IF-THEN structure makes knowledge representation simple and enhances readability.

Modularity: Knowledge is coded in discrete pieces, making it easy to add, modify, or delete information without side effects.

Modifiability: Production rules can be modified to suit different applications.

Knowledge-intensive: The system stores knowledge, and rules are written in a human-readable language, addressing semantic issues.

Production System Characteristics:

Production systems are cognitive architectures used in artificial intelligence to implement search algorithms and emulate human problem-solving skills. They can be categorized based on two characteristics: monotonicity and commutativity.

Monotonicity:

- A monotonic production system is one where applying a rule never prevents another rule from being applied later. Allows simultaneous application of rules without interference.
- A non-monotonic production system is one where applying a rule can prevent other rules from being applied later. Efficiently solves problems without backtracking.

Commutativity:

- A partially commutative production system has the property that if a sequence of rules transforms one state to another, any permutation (reordering) of those rules that satisfies all preconditions will also transform the initial state to the final state
- A non-partially commutative system is one in which the order of rule application matters.

Based on these two characteristics, production systems can be categorized into four types:

Table 2.1 Four Categories of Production Systems

Production System	Monotonic	Non-monotonic
Partially Commutative	Theorem Proving	Robot Navigation
Non-partially Commutative	Chemical Synthesis	Bridge

While all problem types can theoretically be solved by any production system, practical considerations guide the selection of the most suitable system for a given problem.

Partially commutative, monotonic production systems are effective for solving ignorable problems, where backtracking to previous states is unnecessary.

Non-partially commutative systems are useful for problems involving permanent changes, where backtracking is essential to correct mistakes.

Problem Characteristics:

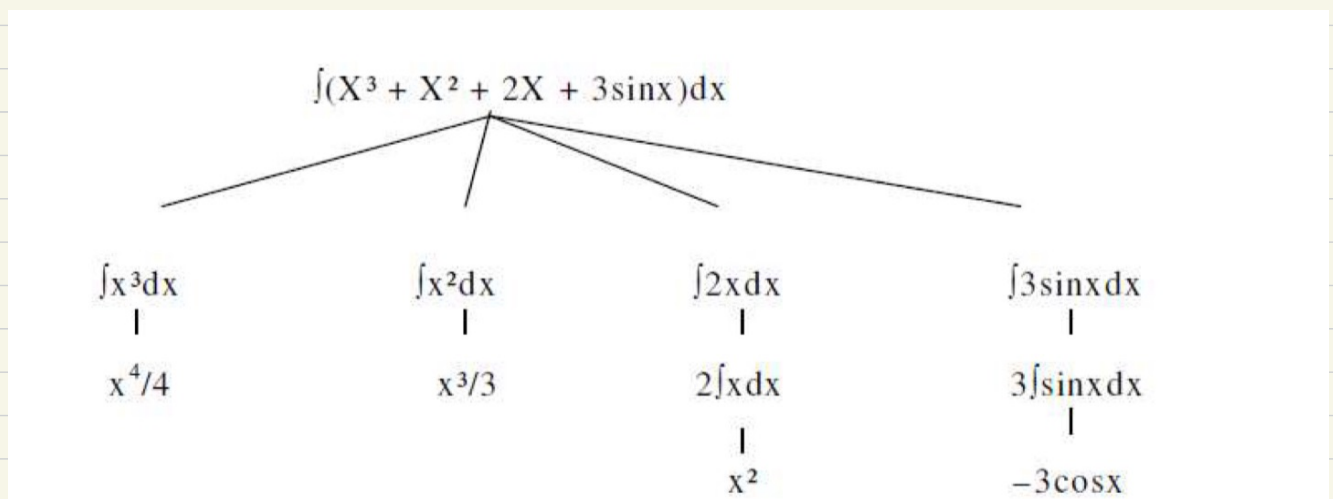
Heuristics play a crucial role in solving complex problems in AI that would otherwise be computationally expensive or intractable. Heuristics are rules of thumb or educated guesses that help guide the problem-solving process in a more efficient manner, exploiting domain-specific knowledge. These heuristics are represented in the form of IF-THEN production rules in production systems.

Many AI problems, especially those that simulate intelligent behavior, make extensive use of heuristic search techniques. The heuristics encoded in the rules represent domain knowledge, while other heuristics define the control strategies that steer the search process itself.

When employing heuristic search for problem-solving, it is essential to analyze the problem characteristics along several dimensions:

1) **Decomposability:** Can the original problem be broken down into a set of smaller, independent subproblems that are easier to solve? If so, applying problem decomposition and solving each component leveraging specific rules can greatly simplify the overall solution.

Suppose to solve the expression is: $\int (X^3 + X^2 + 2X + 3\sin x) dx$



This problem can be solved by breaking it into smaller problems, each of which we can solve by using a small collection of specific rules.

2) Reversibility: Is it possible to undo solution steps if they are found to be unwise or leading to a dead-end?

This gives rise to three problem categories:

a) Ignorable (e.g., theorem proving): Solution steps can be simply ignored without consequences.

b) Recoverable (e.g., 8-puzzle): Solution steps can be undone, and the solver can backtrack to previous states.

c) Irrecoverable (e.g., chess): Once a move is made, it cannot be undone, and the solver must proceed from the current state.

3) Predictability: How predictable is the problem universe? In predictable cases (e.g., 8-puzzle), the outcome of each action is known with certainty, allowing for complete planning. In unpredictable cases (e.g., bridge), plans must be revised as new information emerges.

4) Solution Obviousness: For some problems, an obvious solution may exist without requiring an exhaustive comparison of all possibilities. Recognizing such cases can greatly simplify the search.

5) Solution Type: Will the solution be a final goal state or a path/sequence of actions leading to the goal?

6) Knowledge Role: How much does domain-specific knowledge influence the problem-solving process and guide the search?

7) User Interaction: Does the solution process require interaction with a user, or can it be fully automated?

These characteristics are deeply intertwined. For instance, problem decomposition exemplifies how a complex problem (e.g., evaluating an integral) can be broken down into simpler subproblems (e.g., solving individual integrals) by applying specific rules.

The nature of solution steps (reversible or not) impacts the search strategy. In recoverable cases, backtracking and undoing steps is possible, while in irrecoverable cases, the solver must commit to each move.

For some problems, like querying a knowledge base, there may be multiple valid paths to the solution. If one path successfully leads to the correct answer, there is no need to explore alternative paths, as the goal is to find a solution rather than the specific solution process.

Issues in the Design of Search Programs:

In AI, search algorithms play a crucial role in finding solutions to problems by traversing a search space represented as a tree. However, designing efficient search programs involves addressing several key issues:

1) Tree Traversal:

- The search process involves traversing a tree from the initial state to a goal state.
- Generating nodes in the tree can lead to a large number of unnecessary nodes.
- Effective search algorithms generate only nodes likely to be useful.

2) Search Direction (Forward and Backward Search):

- Search can proceed forward from the initial node to the goal state or backward from the goal state to the initial state.

3) Efficient Rule Matching:

- Selecting applicable rules requires efficient procedures for matching rules against states.

4) Node Representation:

- Representing each node of the search process involves addressing the knowledge representation problem or the frame problem.
- Simple arrays may be sufficient for games like Tic-Tac-Toe, while more complex data structures are needed for other problems.

5) Data Structure Choice (Graph vs. Tree Representation):

- Choosing between graph and tree representation depends on factors like breadth-first or depth-first search and considering duplicate nodes.

6) Duplicate Node Handling:

- Check for duplicate nodes to avoid redundancy in the search process.
- If a new node exists, add it to the graph; otherwise, update the existing node if necessary.

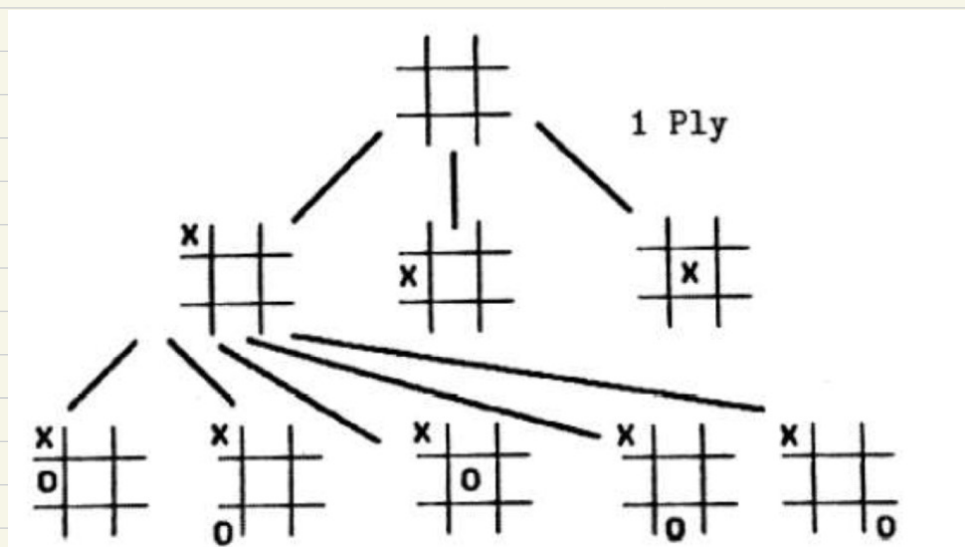
Example: Tic-Tac-Toe

State Space Representation:

- State: Player to move next (X or O) and board configuration (array of 9 cells)
- Operator/Action: Change an empty cell to X or O
- Start State: Empty board, X's turn
- Terminal States: Three X's/O's in a row, or a column or a diagonal

Search Tree:

- Sequence of states formed by legal moves is called a search tree.
- Each level is called a ply.
- The state space may be a graph, but treated as a tree for simplicity (duplicating states).



Solving Problems using Search:

1. Construct a formal description as a state space:

- Define a data structure to represent the state.
- Create a representation for the initial state.
- Implement operators to change the state representation.
- Develop a program to detect terminal states.

2. Choose an appropriate search technique:

- Consider the size and structure of the search space.
- Utilize domain-specific knowledge to guide the search process effectively.

Two Path Problem:

Pathfinding is the process of finding the most efficient route between two nodes in a graph. It's a fundamental problem in computer science and is essential in various applications, including route planning in GPS systems, navigation in video games, and network routing in telecommunications.

Two Primary Problems in Pathfinding:

1) Finding a Path Between Nodes in a Graph:

- This involves determining whether a path exists between two nodes in a graph. It's essential for establishing connectivity and accessibility between different nodes in a graph.

2) Finding the Shortest or Optimal Path:

- Once a path is found, the next step is often to find the shortest or optimal path between the two nodes. The shortest path minimizes some measure, such as distance, cost, or time between the two nodes.

Basic Algorithms for Finding the Best Paths:

1) Breadth-First Search (BFS): Breadth-First Search (BFS) is an algorithm used to explore all the neighboring nodes at the current depth level before moving on to nodes at the next depth level. It is a systematic and efficient way of traversing a graph or a tree data structure. BFS is guaranteed to find the shortest path between two nodes if one exists, but it can be computationally expensive for large graphs or networks.

2) Depth-First Search (DFS): Depth-First Search (DFS) is an algorithm that explores as far as possible along each branch before backtracking. It starts from the source node and explores as far as possible along each branch before backtracking and exploring another branch. DFS is useful for finding a path between two nodes, but it does not guarantee finding the shortest path.

Both BFS and DFS are fundamental algorithms in pathfinding and are often used as building blocks for more advanced pathfinding algorithms, such as A* (A-star), Dijkstra's algorithm, and various heuristic-based approaches.

Predicate Logic:

Predicate Logic in AI serves as a method for describing and modifying assertions about objects and their attributes. It employs predicates, variables, quantifiers, and logical connectives to construct complex statements from simpler ones.

The core components are:

Predicates: These are assertions made about one or more objects. For example, "is blue" is a predicate that states a specific object has the property of being blue.

Variables: These stand in for actual objects, allowing assertions to apply to any object of a particular type. For example, using x to represent any car allows generalizing statements about vehicles.

Quantifiers: These indicate if a statement applies to all objects or at least one object in a domain. The main quantifiers are:

- "For all" ($\forall$): States the assertion is true for all objects in the domain.

E.g. "For all x , x is blue" means every object has the blue property.

- "Exists" ($\exists$): States the assertion is true for at least one object in the domain.

E.g. "Exists x , x is blue" means at least one object has the blue property.

Using predicates, variables and quantifiers, complex logical statements can be constructed like:

"All mammals are warm-blooded" can be represented as $\forall x (\text{Mammal}(x) \rightarrow \text{WarmBlooded}(x))$

This uses the predicates $\text{Mammal}(x)$ and $\text{WarmBlooded}(x)$, the variable x , the universal quantifier $\forall$, and the logical connective $\rightarrow$ (implies).

Other logical connectives used are:

- And ($\wedge$): $\text{Mammal}(x) \wedge \text{WarmBlooded}(x)$ means "x is a mammal and warm-blooded"
- Or ($\vee$): $\text{Mammal}(x) \vee \text{Bird}(x)$ means "x is a mammal or a bird"
- Not ($\neg$): $\neg \text{Mammal}(x)$ means "x is not a mammal"

Representation of Simple Fact:

In artificial intelligence (AI), representing simple facts in logic refers to the process of expressing real-world information or knowledge in a formal, logical format that can be understood and processed by AI systems. This representation allows AI systems to reason about the facts, draw inferences, and make decisions based on the represented knowledge.

Ways to Represent Simple Facts in Logic:

1) Propositional Logic: Propositional logic is a simple way to represent facts using propositions, which are statements that can be either true or false. For example:

- "Marcus is a man" can be represented as $\text{man}(\text{Marcus})$.
- "Plato is a man" can be represented as $\text{man}(\text{Plato})$.
- "All men are mortal" can be represented as $\text{mortal}(\text{men})$.

However, propositional logic fails to capture the relationship between an individual being a man and that individual being mortal. It cannot infer the fact that "Marcus is mortal" and "Plato is mortal" from the given statements.

2) Predicate Logic (First-Order Logic): Predicate logic, also known as first-order logic (FOL), is a more powerful way to represent complex facts using predicates, variables, and quantifiers. It allows for the representation of relationships between objects and their properties.

In predicate logic, facts are represented using predicates and variables. For example:

- "Marcus is a man" can be represented as $\text{Man}(\text{marcus})$.
- "Plato is a man" can be represented as $\text{Man}(\text{plato})$.
- "All men are mortal" can be represented as $\forall x (\text{Man}(x) \rightarrow \text{Mortal}(x))$.

In the last statement, $\forall x$ (read as "for all x") is a universal quantifier, and $\rightarrow$ (read as "implies") is a logical connective. This statement represents the relationship between being a man and being mortal, allowing the inference that "Marcus is mortal" and "Plato is mortal" from the given statements.

Predicate logic also includes other logical connectives like "and" ($\wedge$), "or" ($\vee$), and "not" ($\neg$), which help in constructing more complex logical expressions.

While representing facts in logic, it's important to consider ambiguities in natural language, choose appropriate representations, and include implicit knowledge that might be necessary for effective reasoning.

Computable Predicates:

Computable predicates refer to logical predicates whose truth values can be computed or determined based on some inputs or arguments. These allow representing and reasoning about relationships between objects or values.

1. Simple relationships like greater-than (gt) and less-than (lt) can be expressed as computable predicates:

- $gt(1,0)$ means "1 is greater than 0"
- $lt(0,1)$ means "0 is less than 1"

2. Computable predicates can take functions as arguments too. For example:

- $gt(2+3, 1)$ first computes $2+3=5$, and then checks if $5 > 1$ is true.

3. They can represent facts about objects and their properties:

- $man(Marcus)$ means "Marcus is a man"
- $Pompeian(Marcus)$ means "Marcus was a Pompeian"
- $born(Marcus, 40)$ means "Marcus was born in 40 AD"

4. Using quantifiers like "for all" ($\forall$), general rules can be expressed:

- $\forall x: man(x) \rightarrow mortal(x)$ means "All men are mortal"

5. Complex scenarios can be modeled by combining multiple predicates:

- To check if "Marcus is alive", additional predicates may be needed like:
 - $alive(x,t)$ means "x is alive at time t"
 - $died(x,t)$ means "x died at time t"

6. Deductions can be made by combining known facts using logical rules.

- If Marcus was a Pompeian, and all Pompeians died in 79 AD from a volcano, we can infer Marcus died in 79 AD.

So in conclusion, computable predicates allow specifying relationships, properties and general rules in logic that a reasoning system can automatically compute upon and make deductions.

Resolution:

Resolution is a method used in artificial intelligence (AI) for theorem proving through contradiction. It's employed when we have multiple statements and need to verify a conclusion derived from those statements.

Unification plays a vital role in resolution-based proofs. It involves finding a substitution that makes two expressions identical.

Resolution Steps:

- Step 1: Statement Conversion: Translate all problem statements into first-order logic.
- Step 2: Normal Form Conversion: Convert first-order logic into conjunctive normal form (CNF), facilitating easier manipulation.
- Step 3: Goal Negation: Assert the negation of the desired conclusion or goal as a new statement.
- Step 4: Draw the Resolution Graph: Combine and resolve clauses iteratively, aiming to derive a contradiction. This involves comparing and eliminating complementary literals.

The resolution rule is a single inference rule that operates efficiently on clauses in conjunctive normal form. It works by combining two clauses that contain complementary literals (a literal and its negation) and resolving them to produce a new clause.

It provides a systematic approach to prove or disprove statements, enhancing the reliability of AI systems. Resolution is widely applicable in various domains of AI, including problem-solving, logic programming, and automated reasoning.

↳ Example :

If it is sunny and warm day you will enjoy.

If it is raining you will get wet.

It is a warm day.

It is raining.

It is sunny.

Goal :→ You will enjoy.

Sol :→ ① Convert facts into First Order Logic (FOL).

a) If it is sunny and warm day you will enjoy.

FOL :→ Sunny $\wedge$ Warm $\rightarrow$ enjoy ($\wedge$ means AND, $\rightarrow$ means implies)

b) If it is raining you will get wet.

FOL :→ raining $\rightarrow$ wet

c) It is a warm day.

FOL :→ Warm

d) It is raining
FOL $\rightarrow$ raining

e) It is sunny.
FOL $\rightarrow$ sunny

② Convert FOL into Conjunctive Normal Form (CNF).

a) $\neg (\text{Sunny} \wedge \text{Warm}) \vee \text{enjoy}$
or $\neg \text{Sunny} \vee \neg \text{Warm} \vee \text{enjoy}$

$\rightarrow$ we have to eliminate the ' $\rightarrow$ '
 $a \rightarrow b$ can be written as $\neg a \vee b$
negate a OR b

b) $\neg \text{raining} \vee \text{wet}$

For the rest of the statements (i.e., c, d, e) no need to change as they already are simple statements.

③ Negate The statement to be proved.

The statement to be proved is $\rightarrow \text{enjoy}$
and its negate will be $\neg \text{enjoy}$

④ Draw the resolution graph.
First we have to take the contradiction.

$\neg \text{enjoy}$
Now in our statements wherever the contradiction of $\neg \text{enjoy}$ occurs we will consider that statement.

The first statements is the one to be considered

$\neg \text{enjoy}$ $\neg \text{Sunny} \wedge \neg \text{Warm} \vee \text{enjoy}$
Now, we will apply unification method on these two.

~~$\neg \text{enjoy}$~~ ~~$\neg \text{Sunny} \wedge \neg \text{Warm} \vee \text{enjoy}$~~
 $\neg \text{Sunny} \wedge \neg \text{Warm}$

Now in our statements wherever the contradiction of $\neg \text{Warm}$ occurs we will consider that statement.

$\neg \text{Sunny} \wedge \neg \text{Warm}$ ~~warm~~ $\leftarrow$ from statement ②.
 $\neg \text{Sunny}$

Now in our statements wherever the contradiction of $\neg \text{Sunny}$ occurs we will consider that statement.

~~$\neg \text{Sunny}$~~ ~~Sunny~~ $\rightarrow$ from statement ②
 $\{ \} \rightarrow \text{Contradiction}$

We got an empty statement which is a contradiction
So, the goal statement is proved.

Example : Consider the following knowledge base:

1. *The-humidity-is-high* $\vee$ *the-sky-is-cloudy*.
2. If *the-sky-is-cloudy* then *it-will-rain*
3. If *the-humidity-is-high* then *it-is-hot*.
4. *it-is-not-hot*

and the goal : *it-will-rain*. Prove by resolution theorem that the goal is derivable from the knowledge base.

Proof: Let us first denote the above clauses by the following symbols.

$p = \textit{the-humidity-is-high}$, $q = \textit{the-sky-is-cloudy}$, $r = \textit{it-will-rain}$, $s = \textit{it-is-hot}$. The Conjunctive Normal Form (CNF) from the above clauses thus become :

1. $p \vee q$
2. $\neg q \vee r$
3. $\neg p \vee s$
4. $\neg s$

and the negated goal = $\neg r$. Set S thus includes all these 5 clauses. Now by resolution algorithm, we construct the solution by a tree. Since it terminates with a null clause, the goal is proved.

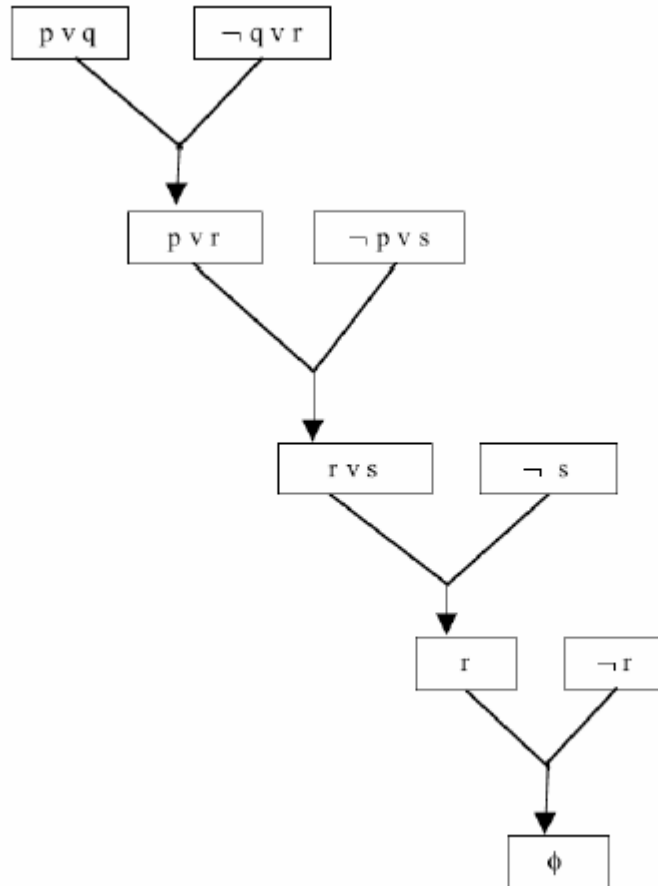


Fig. 1: The resolution tree to prove that it-will-rain.

Example :

"All people who are not poor and are smart are hippy. Those people who read are not stupid. John can read are is wealthy. Happy people have exciting lives. Can anyone be found with an exciting life?"

Sol.:

a) First change the sentences to predicate form:

We assume $\forall X (\text{smart}(X) \equiv \text{stupid}(X))$ and $\forall Y (\text{wealthy}(Y) \equiv \text{poor}(Y))$,
and get:

$$\forall X (\neg \text{Poor}(X) \wedge \text{smart}(X) \Rightarrow \text{happy}(X))$$

$$\forall Y (\text{read}(Y) \Rightarrow \text{smart}(Y))$$

$$\text{read}(\text{john}) \wedge \neg \text{poor}(\text{john})$$

$$\forall Z (\text{happy}(Z) \Rightarrow \text{exciting}(Z))$$

The negation of the conclusion is:

$$\neg \exists W (\text{exciting}(W))$$

b) These predicate calculus expressions for the happy life problem are transformed into the following clauses:

$$\text{poor}(X) \vee \neg \text{smart}(X) \vee \text{happy}(X)$$

$$\neg \text{read}(Y) \vee \text{smart}(Y)$$

$$\text{read}(\text{john})$$

$$\neg \text{poor}(\text{john})$$

$$\neg \text{happy}(Z) \vee \text{exciting}(Z)$$

$$\neg \text{exciting}(W)$$

The resolution refutation for this example is found in Figure 2.

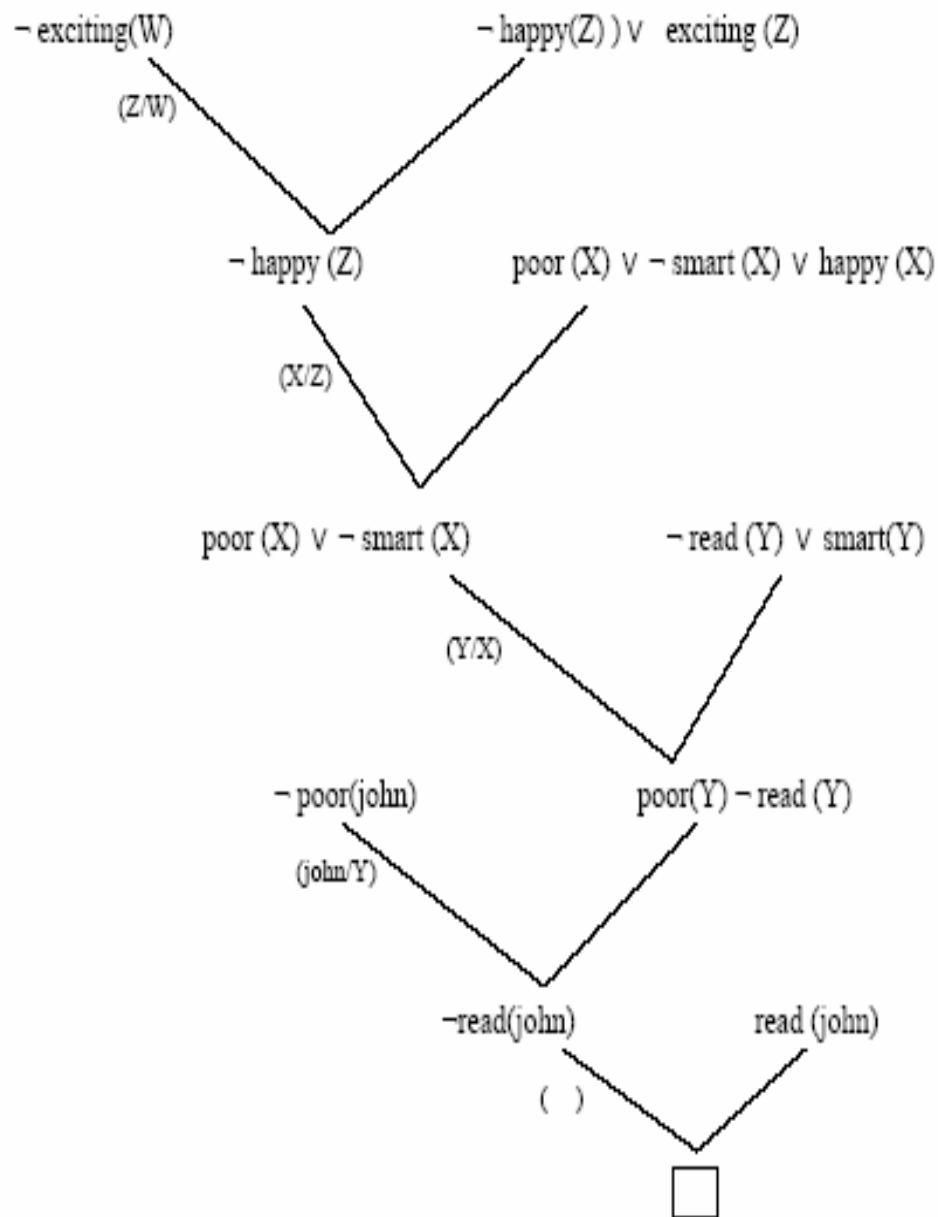


Figure (2): Resolution prove for the "exciting life" problem.

→ Natural Deductions :→ It is a kind of proof calculus in which logical reasoning is expressed by inference rules closely related to the natural way of reasoning.

Inference rules may be stated as below :→

Rule Name	Symbol	Rule	Description
Introducing ($\wedge$)	($I: \wedge$)	If $A_1, \dots, A_n$ then $A_1 \wedge A_2 \wedge \dots \wedge A_n$	If $A_1, A_2, \dots, A_n$ are true, then conjunction $A_1 \wedge A_2 \wedge \dots \wedge A_n$ is also true.
Eliminating ($\wedge$)	($E: \wedge$)	If $A_1 \wedge A_2 \wedge \dots \wedge A_n$ then A_i ($1 \leq i \leq n$)	If $A_1 \wedge A_2 \wedge \dots \wedge A_n$ are true then A_i is also true.

Convert to clause form: FOL->CNF

Convert the following statement to clause form:

$$\begin{aligned} & \forall x [B(x) \rightarrow (\exists y [Q(x,y) \wedge \neg P(y)] \\ & \wedge \neg \exists y [Q(x,y) \wedge Q(y,x)] \\ & \wedge \forall y [\neg B(y) \rightarrow \neg E(x,y)])] \end{aligned}$$

1- Eliminate the implication ($\rightarrow$)

$$\begin{aligned} E1 \rightarrow E2 &= \neg E1 \vee E2 \\ \forall x [\neg B(x) \vee (\exists y [Q(x,y) \wedge \neg P(y)] \\ & \wedge \neg \exists y [Q(x,y) \wedge Q(y,x)] \\ & \wedge \forall y [\neg (\neg B(y)) \vee \neg E(x,y)])] \end{aligned}$$

2- Move the negation down to the atomic formulas (by using the following rules)

- $\neg (P \wedge Q) \equiv \neg P \vee \neg Q$
- $\neg (P \vee Q) \equiv \neg P \wedge \neg Q$
- $\neg (\neg (P)) \equiv P$
- $\neg \forall x (P(x)) \equiv \exists x (\neg P(x))$
- $\neg \exists x (P(x)) \equiv \forall x (\neg P(x))$

$$\begin{aligned} & \forall x [\neg B(x) \vee (\exists y [Q(x,y) \wedge \neg P(y)] \\ & \wedge \forall y [\neg Q(x,y) \vee \neg Q(y,x)] \\ & \wedge \forall y [B(y) \vee \neg E(x,y)])] \end{aligned}$$

3- Purge existential quantifiers

The function that is eliminate the existential are called “ Skolem function”

$$\begin{aligned} & \forall x [\neg B(x) \vee ([\neg Q(x, f(x)) \wedge \neg P(f(x))] \\ & \wedge \forall y [\neg Q(x,y) \vee \neg Q(y,x)] \\ & \wedge \forall y [B(y) \vee \neg E(x,y)])] \end{aligned}$$

4- Rename variables, as necessary, so that no two variables are the same.

$$\begin{aligned} & \forall x [\neg B(x) \vee ([\neg Q(x, f(x)) \wedge \neg P(f(x))] \\ & \wedge \forall y [\neg Q(x,y) \vee \neg Q(y,x)] \\ & \wedge \forall z [B(z) \vee \neg E(x,z)])] \end{aligned}$$

5- Move the Universal quantifiers to the left of the statement.

$$\begin{aligned} & \forall x \forall y \forall z [\neg B(x) \vee ([\neg Q(x, f(x)) \wedge \neg P(f(x))] \\ & \wedge [\neg Q(x,y) \vee \neg Q(y,x)] \\ & \wedge [B(z) \vee \neg E(x,z)])] \end{aligned}$$

6- Move the disjunction down to the literals, using distributive laws

$$\begin{aligned} E1 \vee (E2 \wedge E3 \wedge E4 \wedge \dots) & \equiv (E1 \vee E2) \wedge (E1 \vee E3) \wedge \dots \\ E1 \wedge (E2 \vee E3 \vee E4 \vee \dots) & \equiv (E1 \wedge E2) \vee (E1 \wedge E3) \vee \dots \end{aligned}$$

$$\begin{aligned} & \forall x \forall y \forall z [(\neg B(x) \vee (\neg Q(x, f(x)) \wedge \neg P(f(x)))) \\ & \wedge [\neg B(x) \vee \neg Q(x,y) \vee \neg Q(y,x)] \\ & \wedge [\neg B(x) \vee B(z) \vee \neg E(x,z)]] \end{aligned}$$

$$\begin{aligned} & \forall x \forall y \forall z [(\neg B(x) \vee (\neg Q(x, f(x)) \\ & \wedge (\neg B(x) \vee \neg P(f(x)))) \\ & \wedge (\neg B(x) \vee \neg Q(x,y) \vee \neg Q(y,x)) \\ & \wedge (\neg B(x) \vee B(z) \vee \neg E(x,z)))] \end{aligned}$$

7- Eliminate the conjunctions

$$\forall x \ [\neg B(x) \vee (\neg Q(x, f(x))]$$

$$\forall x \ [\neg B(x) \vee \neg P(f(x))]$$

$$\forall x \ \forall y \ [\neg B(x) \vee \neg Q(x,y) \vee \neg Q(y,x)]$$

$$\forall x \ \forall z \ [\neg B(x) \vee B(z) \vee \neg E(x,z)]$$

8- Rename all the variables, as necessary, so that no two variables are the same.

$$\forall x \ [\neg B(x) \vee (\neg Q(x, f(x))]$$

$$\forall w \ [\neg B(w) \vee \neg P(f(w))]$$

$$\forall u \ \forall y \ [\neg B(u) \vee \neg Q(u,y) \vee \neg Q(y,u)]$$

$$\forall a \ \forall z \ [\neg B(a) \vee B(z) \vee \neg E(a,z)]$$

9- Purge the universal quantifiers.

$$\neg B(x) \vee (\neg Q(x, f(x))$$

$$\neg B(w) \vee \neg P(f(w))$$

$$\neg B(u) \vee \neg Q(u,y) \vee \neg Q(y,u)$$

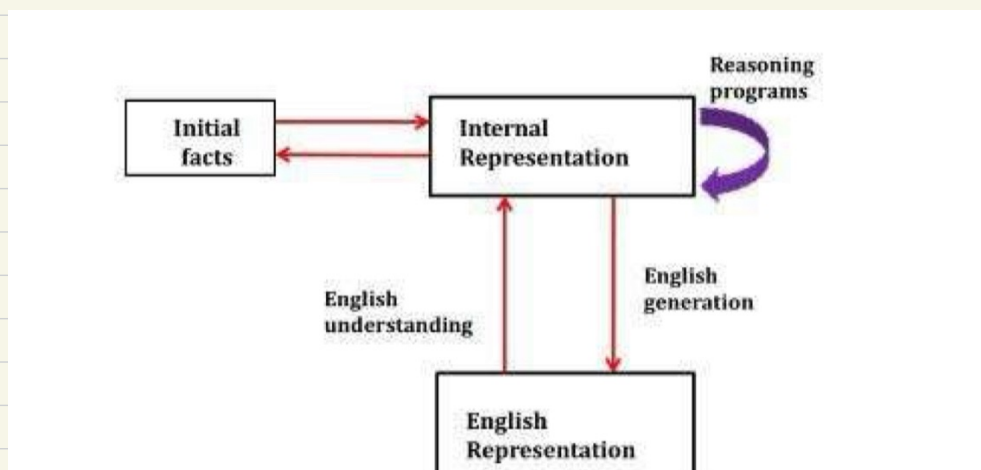
$$\neg B(a) \vee B(z) \vee \neg E(a,z)$$

Representations and Mappings

In Artificial Intelligence (AI), solving complex problems requires a large amount of knowledge and mechanisms for manipulating that knowledge to create solutions. Knowledge and Representation play distinct but central roles in an intelligent system.

Knowledge and Representation:

- Knowledge is a description of the world. It determines a system's competence by what it knows.
- Representation is the way knowledge is encoded. It defines a system's performance in doing a task.
- Different types of knowledge require different kinds of representation.



Knowledge representation models in AI are often based on:

- Logic
- Rules
- Frames
- Semantic Networks

Types of Knowledge:

1) Tacit Knowledge

- Informal or implicit knowledge
- Exists within a human being
- Embodied and difficult to articulate formally
- Difficult to communicate or share
- Hard to steal or copy
- Drawn from experience, action, and subjective insight

2) Explicit Knowledge:

- Formal type of knowledge
- Exists outside a human being
- Embedded and can be articulated formally
- Can be shared, copied, processed, and stored
- Easy to steal or copy
- Drawn from artifacts such as principles, procedures, processes, and concepts

Facts and Representations:

- Facts are truths in some relevant world that we want to represent.
- Representations of facts are in a chosen formalism that can be manipulated by programs.

Knowledge Representation Framework:

The process of knowledge representation involves the following steps:

1. Informal formulation of the problem in natural language.
1. Formal representation of the problem in a language that computers can understand.
3. Computer processing and computation of an answer.
4. Representation of the output in an informally described solution that the user can understand.

Knowledge Representation Approaches

1. Relational Knowledge:

- The simplest way to represent declarative facts is using a set of relations, similar to databases.
- Facts about objects are stored in a tabular format with columns representing attributes.
- This representation provides little opportunity for inference, but facts can be queried based on attributes.

2. Inheritable Knowledge:

- Knowledge elements inherit attributes from their parents in a hierarchical structure.
- Objects or elements of specific classes inherit attributes and values from more general classes.
- Classes are organized in a generalized hierarchy, often represented as semantic networks or frames.
- Inheritance is a powerful form of inference, but it needs to be augmented with additional inference mechanisms.

3. Inferential Knowledge:

- This knowledge generates new information from given information through analysis and inference.
- Predicate logic (mathematical deduction) is used to infer new values or relations from a set of attributes.
- Knowledge is represented as formal logic statements, e.g., "All dogs have tails" $\forall x: \text{dog}(x) \rightarrow \text{hastail}(x)$.
- Advantages: Strict rules, ability to derive new facts, verify truths, guaranteed correctness, and availability of inference procedures.

4. Procedural Knowledge:

- Knowledge is represented as procedures or programs that encode how to perform specific tasks or actions.
- Knowledge is embedded in the control information within the procedures themselves.
- Examples: Computer programs, directions, recipes.
- Advantages: Ability to represent heuristic or domain-specific knowledge, facilitate extended logical inferences and model side effects of actions.
- Disadvantages: May not represent all cases (completeness issue), not all deductions may be correct (consistency issue), sacrificed modularity, and cumbersome control information.

Knowledge Representation Issues:

1. *Important Attributes:*

- Two attributes that are generally significant are "instance" and "isa" as they support property inheritance.

2. *Relationships among Attributes:*

- There are important relationships among object attributes, such as:
- Inverse relationships for consistency checking.
- Existence in an "isa" hierarchy for generalization and specialization.
- Techniques for reasoning about attribute values not explicitly given.
- Single-valued attributes that can have only one value.

3. *Choosing Granularity:*

- Determining the appropriate level of detail for representing knowledge is crucial.
- Representing knowledge at a high level may not be adequate for inference, while low-level primitives may require excessive storage.
- The choice of granularity depends on the specific problem and the desired level of inference.

4. *Set of Objects:*

- Certain properties may be true for objects as members of a set but not as individuals.
- Representing sets of objects is more efficient than associating properties with each individual element.
- Sets can be represented using the universal quantifier in logical representations or as nodes in hierarchical structures with inheritance.

5. *Finding the Right Structure:*

- Given a large amount of knowledge, accessing the relevant parts when needed is a challenge.
- This involves selecting an initial structure and revising it if it turns out to be inappropriate.

Frame Problem

The frame problem in artificial intelligence pertains to challenges faced when using first-order logic (FOL) to convey facts about real-world robots. FOL limits predicate reference to a single subject, necessitating additional axioms to describe robot environments accurately. Coined in 1969 by McCarthy and Hayes, it served as a foundation for understanding knowledge representation challenges in AI. Philosophically, it denotes the need to revise ideas in light of new information.

Issues Related to the Frame Problem:

- 1) Qualification Problem: Uncertainty in the effectiveness of rules and the inability to adapt to changing circumstances.
- 2) Representational Problem: Difficulty in accurately representing present environmental truths.
- 3) Inferential Problem: Challenges in examining how the world is judged and inferring changes.
- 4) Ramification Problem: Explains how behavior leads to environmental changes.
- 5) Predictive Problem: Uncertainty in predicting positive environmental changes.

Solutions Proposed:

- 1) Non-Deductive Approach: Mimics human thought processes but lacks success in replicating human cognition.
- 2) Deductive Approach: Utilizes axioms and predicate calculus to draw conclusions, albeit only in simple instances.
- 3) Frames & Scripts Approach (Minsky & Schank): Categorizes and segments the world into frames or scripts to develop routines for specific scenarios.
- 4) Experience Approach (Hume): Plans actions based on assumptions, learns from mistakes, and gains experience.
- 5) Ad Hoc: Incorporates probability into decision-making to predict success probabilities.

Semantic Networks

Semantic networks are a form of knowledge representation in Artificial Intelligence (AI) that provides an alternative to predicate logic. In semantic networks, knowledge is represented graphically using nodes and arcs (links).

Components:

- Nodes represent objects or concepts.
- Arcs (links) describe the relationships between the objects.

Relations:

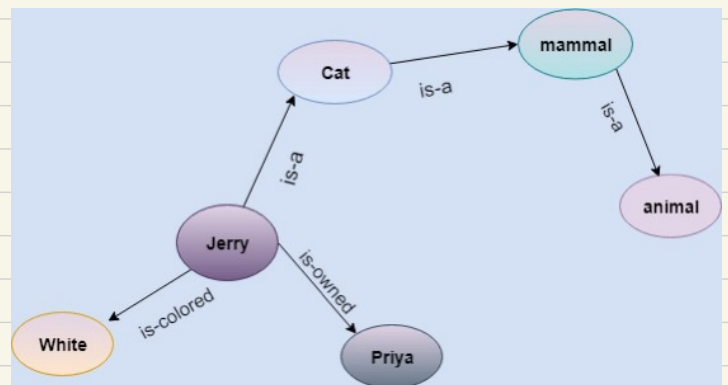
Semantic networks typically use two main types of relations:

- 1) IS-A Relation (Inheritance): This relation represents a hierarchical relationship between objects, where one object is a specific instance of a more general object.
- 2) Kind-of Relation: This relation represents a categorical relationship between objects, where one object is a member of a particular category or class.

Example:

Consider the following statements:

- 1 Jerry is a cat.
- 2 Jerry is a mammal.
- 3 Jerry is owned by Priya.
- 4 Jerry is brown-colored.
- 5 All mammals are animals.



Advantages of Semantic Networks:

- 1) Natural representation of knowledge.
- 2) Convey meaning transparently.
- 3) Simple and easily understandable.

Drawbacks of Semantic Networks:

- 1) Computational time: Traversing the entire network to answer queries can be time-consuming, and the solution may not exist in the network.
- 2) Limited scale: Modeling human-like memory with vast numbers of neurons and connections is not practical.
- 3) Lack of quantifiers: Semantic networks do not have equivalent quantifiers (e.g., for all, for some, none).
- 4) No standard link names: There is no standard definition for the names of links (arcs).
- 5) Lack of intelligence: Semantic networks depend on the creator of the system and are not inherently intelligent.

Non-monotonic reasoning:

Non-monotonic reasoning is a type of reasoning used in artificial intelligence systems where conclusions can be revised or retracted when new information is added.

In regular monotonic logic, once a conclusion is reached from a set of premises, that conclusion must hold even if new premises are added later. Monotonic logic doesn't allow for revising conclusions.

However, non-monotonic logic allows for more flexible reasoning that is closer to how humans reason in the real world. We frequently have to update our beliefs and conclusions when we learn new contradictory information.

A classic example is:

Premise 1: Birds can fly

Premise 2: Tweety is a bird Conclusion:

Therefore, Tweety can fly

But if we add a new premise:

Premise 3: Tweety is a penguin

We have to retract our initial conclusion that Tweety can fly, since penguins are a type of bird that cannot fly. The addition of new information caused us to revise our beliefs.

Advantages:

- **Real-world Applications:** Non-monotonic reasoning is suitable for real-world systems like robot navigation, where uncertainty and dynamic environments are common.
- **Probabilistic Reasoning:** It allows for the incorporation of probabilistic facts and assumptions, making it adaptable to various scenarios.

Disadvantages:

- **Invalidation of Previous Conclusions:** New information may invalidate previous conclusions, making it challenging to maintain consistency.
- **Limited Use in Theorem Proving:** Non-monotonic reasoning is not suitable for theorem proving tasks where maintaining the validity of conclusions is crucial.

Monotonic VS Non-Monotonic Reasoning:

Monotonic Reasoning	Non-Monotonic Reasoning
Monotonic Reasoning is the process which does not change its direction or can say that it moves in the one direction.	Non-monotonic Reasoning is the process which changes its direction or values as the knowledge base increases.
Monotonic Reasoning deals with very specific type of models, which has valid proofs.	Non-monotonic reasoning deals with incomplete or not known facts.
The addition in knowledge won't change the result.	The addition in knowledge will invalidate the previous conclusions and change the result.
In monotonic reasoning, results are always true, therefore, set of prepositions will only increase.	In non-monotonic reasoning, results and set of prepositions will increase and decrease based on condition of added knowledge.
Monotonic Reasoning is based on true facts.	Non-monotonic Reasoning is based on assumptions.
Deductive Reasoning is the type of monotonic reasoning.	Abductive Reasoning and Human Reasoning is a non-monotonic type of reasoning.

Implementation issues for non-monotonic reasoning:

Non-monotonic reasoning allows for revising conclusions when new contradictory information is added. However, implementing it comes with some challenges:

Weak Slot and Filler Structures:

- Semantic networks often lack the structure to properly handle non-monotonic reasoning.
- More structured representations called "frames" are better suited.

Frames as Sets and Instances:

- Frames represent either a class (set) or an instance (element of a class).
- Classes have two types of attributes: their own, and those inherited by set elements.
- Sometimes the line between a set and an instance is blurred (e.g. a team can be a set of players or an instance of a class).

Metaclasses:

- To differentiate regular classes from classes of classes, the concept of metaclasses is used.
- The most basic metaclass is CLASS, representing the set of all classes.
- Metaclasses introduce properties like cardinality that get inherited.

Relating Classes:

- Beyond subclass relationships, classes can relate in other ways:
 - Mutually disjoint: guaranteed no common elements
 - Is-covered-by: the class is the union of a set of subclasses

Slots as Frames:

- Rather than simple attributes, slots need to be represented explicitly as frames themselves.
- Slot frames describe properties like domains, value constraints, defaults, inheritance rules.
- Slot frames are organized into an inheritance hierarchy like other frames.
- Formally, a slot is a relation mapping elements of a domain class to possible values.

Slots as Relations:

- A slot can be viewed as a set of ordered pairs forming a relation.
- One relation can be a subset/specialization of another relation.
- The set of all slots forms a metaclass called SLOT, with instances as specific slots.

Depth-first Search (DFS):

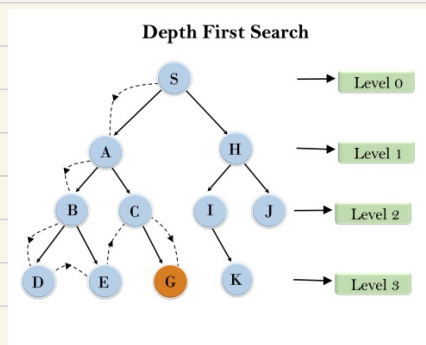
Depth-first search (DFS) is a search strategy in artificial intelligence where the search extends the current path as far as possible before backtracking to explore other alternatives. DFS does not guarantee finding the optimal solution. It efficiently explores large search spaces, often reaching a satisfactory solution more rapidly than breadth-first search.

Algorithm:

1. Start at the root node.
2. Pick one of the node's child nodes to explore recursively with DFS.
3. When you can't go any further from that node (hit a leaf), backtrack to the most recent node that has unexplored children.
4. Repeat steps 2 and 3 until the target node is found or the entire tree/graph is exhausted.

So it dives down vertically as far as possible before backtracking horizontally to explore other branches. This is unlike breadth-first search which explores all nodes at the current depth before moving deeper.

Example:



Advantages:

- Uses limited memory as it only needs to store the current path being explored
- Easy to implement and can be recursive
- Can quickly find a solution path, if one exists close to the root

Disadvantages:

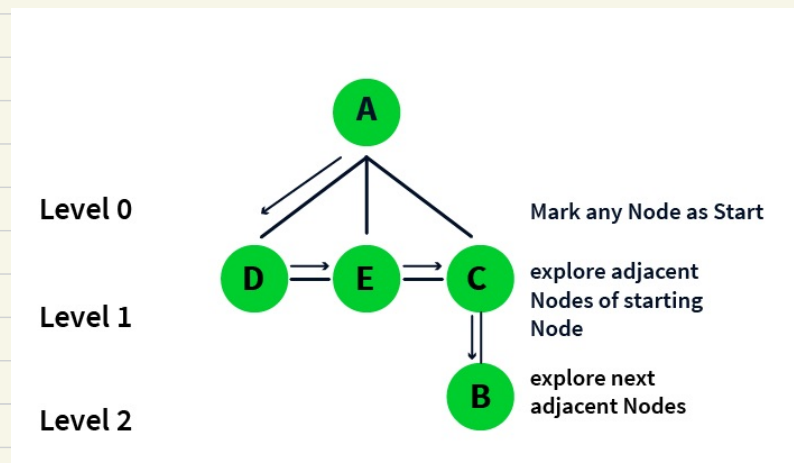
- Doesn't always find the shortest path to the solution
- Can get stuck exploring a deep branch that doesn't lead to a solution
- Doesn't work well for infinite trees/graphs as it may never backtrack

Breadth-First Search (BFS):

BFS is a search strategy used in AI and computer science to explore all nodes at the current depth level before moving on to nodes at the next depth level. It searches horizontally outward from the starting node, ensuring all possibilities at each level are explored before going deeper.

Algorithm:

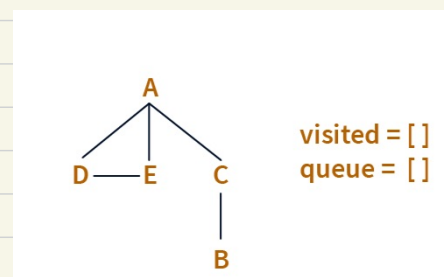
- 1) Initialization: Start with the starting node, enqueue it into a queue, and mark it as visited.
- 2) Exploration: While the queue is not empty:
 - Dequeue a node and visit it.
 - Enqueue all unvisited neighbors of the dequeued node and mark them as visited.
- 3) Termination: Repeat until the queue is empty.



Example:

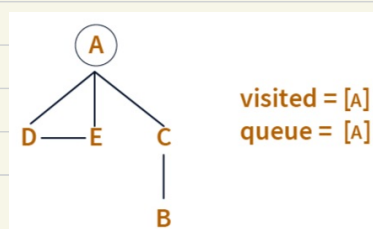
Let's see how to use a Breadth-First Search from Node A.

We need to use two data structures a Queue (for FIFO property) and a Set visited (to mark the visited nodes).



Step: 1

We pick A as the starting point and add A to the Queue. To prevent cycles, we also mark A as visited (by adding it to the visited set).

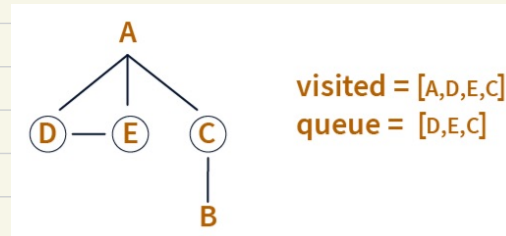


Step: 2

We remove the head of the Queue (i.e. A now). The Node was First In (inserted first) in the Queue.

We process A and pick all its neighbours that have not been visited yet (i.e., not in the visited set). Those are D, C, and E.

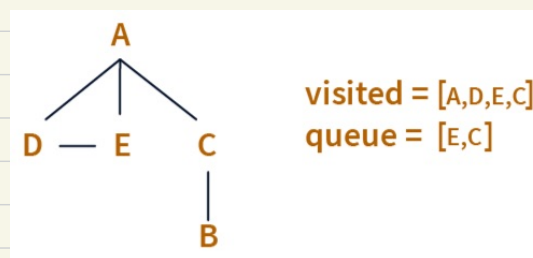
We add D, C, and E to the Queue and these to the visited set.



Step :3

Next, we pull the head of the Queue, , i.e. D.

We process D and consider all neighbours of D, which are A and E, but since both A and E are in the visited set, we ignore them and move forward.

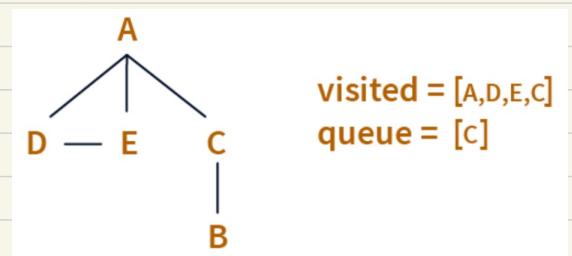


Step :4

Next, we pull the head of the Queue, i.e. E.

We process E.

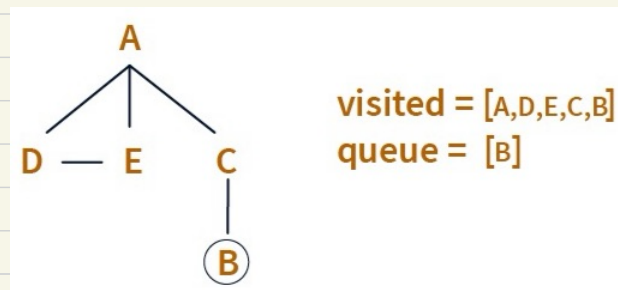
Then we need to consider all neighbours of E, which are A and D, but since both A and D are in the visited set, we ignore them and move forward.



Next, we pull the head of the Queue, i.e. C.

We process C.

Then we consider all neighbours of C, which are A and B. Since A is in the visited set, we ignore it. But as B has not yet been visited, we visit B and add it to the Queue.



Step 5:

Finally, we pick B from Queue, and its neighbour is C, which has already visited. We have nothing else in Queue to process, so we are done traversing the graph.

So the order in which we processed/explored the elements are: A, D, E, C, B which is the Breadth-First Search of the above Graph.

So we see that the Breadth-First Search relies on 2 other data structures i.e. A queue and a Visited Set (or Arrays).

Advantages:

- Guarantees finding the shortest path/solution if one exists.
- Finds minimal solution with least steps if multiple solutions exist.

Disadvantages:

- High memory usage to store all nodes at each level.
- Slow if solution is far away from the root node.

Bayes' Theorem:

Bayes' theorem is a mathematical formula used to calculate the conditional probability of event A given the occurrence of event B. It's named after Thomas Bayes, an 18th-century mathematician. It allows updating beliefs or probabilities as new evidence is obtained. It is utilized in various fields including artificial intelligence and machine learning.

The Formula $P(A|B) = (P(B|A) * P(A)) / P(B)$

Where:

- $P(A|B)$ is the conditional probability of A given B has occurred.
- $P(B|A)$ is the conditional probability of B given A has occurred.
- $P(A)$ and $P(B)$ are the independent probabilities of A and B occurring.

Special Considerations:

Prior Probabilities: Bayes requires an initial probability estimate which should be updated with new evidence.

Independence: The events should be independent for simplicity, but adjustments can be made if dependent.

Data Quality: Accurate data on prior/conditional probabilities is crucial for meaningful results.

Interpretation: Results show conditional probability, not causal relationship.

Certainty Factors:

A model to represent and manipulate uncertain knowledge in early rule-based expert systems (1980s). It attaches a certainty measure/factor to premises and conclusions in rules. It allows reasoning with uncertain information.

Example Rule with Certainty Factor:

If A (with certainty x) then B (with certainty $f(x)$)

- x is the certainty factor for premise A
- $f(x)$ calculates the certainty for the conclusion B based on x

Dealing with Uncertainty in Rule-Based Systems:

- Rules often deal with non-deterministic relationships and uncertain information.
- Certainty factors are aimed to quantify and propagate this uncertainty.

Common approaches:

- Adding certainty factors to rules
- Using Dempster-Shafer belief functions
- Including fuzzy logic

Limitations of Certainty Factors:

- Criticized as being ad-hoc and not well-founded statistically.
- Combining uncertainties is not just a local rule operation, but depends on global context.
- Can lead to incorrect conclusions when uncertainties are compounded.

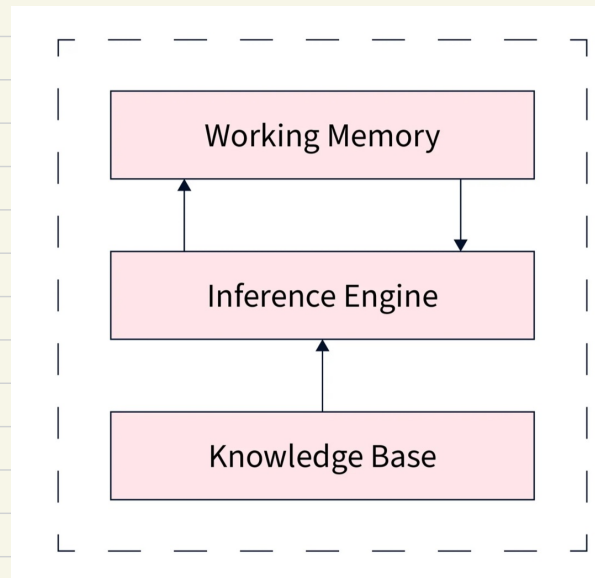
Replacement by Bayesian Networks:

- Certainty factor models have been largely replaced by Bayesian Belief Networks (BBNs).
- BBNs provide a more expressive and principled way to represent and reason with uncertainties.
- Build global probabilistic models instead of isolated certainty propagation in rules.

Rule-based Systems:

A rule-based system in AI relies on predetermined rules to determine actions or decisions based on given inputs and conditions. These rules are formulated based on various circumstances and actions, providing a structured framework for decision-making. It uses logical rules like "IF condition THEN action/conclusion" to reason and produce outputs.

Examples: Expert systems, decision support systems, chatbots.



Key Characteristics:

Simple: Human-readable rules for transparency and ease of maintenance.

Deterministic: Same input always produces the same output (predictable).

Explainable: Rules are explicit, so reasoning is interpretable.

Scalable: Can handle large rule sets and data volumes when implemented properly.

Modular: Rules can be added/modified independently.

How They Work:

1) Input data is matched against the rule conditions (IF part).

2) If a rule's conditions are satisfied, the corresponding actions/conclusions are triggered (THEN part).

3) If no rules apply, a default output may be given, or the system may ask for more input.

Core Components:

- 1) Knowledge Base: Stores the rules and domain knowledge.
- 2) Inference Engine: Applies logical reasoning to match rules and derive conclusions.
- 3) Working Memory: Holds current data and inferred facts during reasoning.
- 4) User Interface: Allows user input and displays outputs.
- 5) Explanation Facility: Provides justifications for system's decisions.
- 6) Database: Stores relevant data for the system.
- 7) External Interface: Enables integration with other systems.

Creating a Rule-Based System:

- 1) Define the problem to be solved.
- 2) Establish the rule set based on domain expertise or data analysis.
- 3) Implement rules in a rule engine or development environment.
- 4) Test and evaluate the system's performance, then iterate as needed.

Examples:

- Medical diagnosis based on symptoms and test results.
- Fraud detection based on transaction patterns.
- Quality control for manufacturing defects.
- Decision support for investment or purchasing decisions.

Advantages:

- Transparency and explainability due to explicit rules.
- Flexibility to modify or add new rules.
- Scalability to handle large rule sets and data volumes.
- Deterministic and consistent decision-making.

Limitations:

- Complexity in managing large rule bases.
- Reliance on complete and accurate predefined rules (no learning).
- Limited ability to handle uncertainty or unforeseen scenarios.
- Potential knowledge acquisition bottleneck when eliciting rules from experts.

Bayesian Networks/ Bayesian Belief Network (BBNs):

- Probabilistic graphical models to represent and reason about uncertain relationships between variables.
- BBNs use directed acyclic graphs (DAGs) to visually depict causal/dependence relationships.
- It leverages Bayes' theorem to perform probabilistic inference and update beliefs based on new evidence.

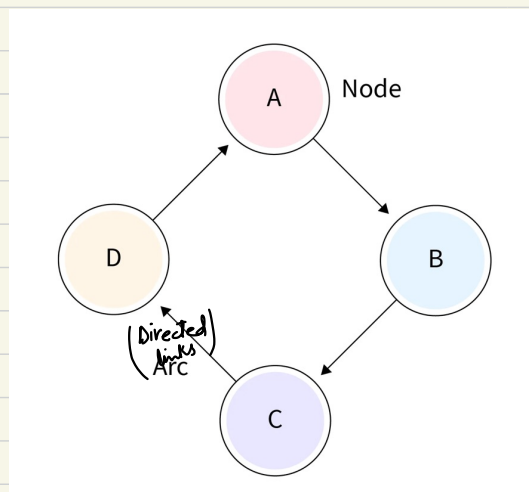
Bayesian Network can be used for building models from data and experts opinions, and it consists of two parts:

1) Directed Acyclic Graph (DAG):

- Nodes represent random variables
- Edges show conditional dependencies/independencies
- Direction indicates causality

2) Conditional Probability Tables (CPTs):

- Each node has a CPT specifying its probability given its parent nodes
- Quantifies the strengths of the dependencies



In the above diagram, A, B, C, and D are random variables represented by the nodes of the network graph.

If we are considering node B, which is connected with node A by a directed arrow, then node A is called the parent of Node B.

Node C is independent of node A.

The Bayesian network has mainly two components:

1) Causal Component (DAG structure):

- Visually represents causal relationships between variables
- Provides insights into how variables influence each other

2) Numerical Component (CPTs):

- Contains conditional probability distributions
- Enables probabilistic computations and inference

Bayesian network is based on Joint probability distribution and conditional probability.

Joint Probability Distribution:

- Describes the probability of all configurations of network variables.
- The product of all CPTs gives the joint probability over all variables
- Allows querying any subset of variables given evidence
- Used for inference, parameter estimation, model selection, and prediction.

Applications:

- Diagnostic systems (e.g. medical diagnosis)
- Predictive modeling (e.g. machine health monitoring)
- Decision support systems
- Causal reasoning and analysis
- Data mining and knowledge discovery

Advantages:

- Intuitive visual representation of variable relationships
- Ability to perform coherent reasoning under uncertainty
- Combining domain knowledge and data for learning
- Explanation capabilities by tracing evidential reasoning

Disadvantages:

- Constructing networks can be laborious for large domains
- Inference can be computationally expensive for complex models
- Dealing with missing data or non-causal relationships can be challenging

Dempster-Shafer Theory (DST):

A mathematical theory of evidence to represent and reason with uncertainty. It extends traditional probability theory to handle evidence about sets of events, not just single events. It also allows more nuanced and flexible modeling of uncertain information without extra assumptions.

It was developed by Arthur P. Dempster and Glenn Shafer in 1967. It addresses limitations of Bayesian theory in handling single evidence and inability to describe ignorance.

Need:

- Extends traditional probability theory to handle sets of events in a finite discrete space.
- Represents uncertainty through belief functions, allowing for nuanced representation of incomplete information and conflicting evidence.
- While Bayesian theory deals with single evidence, DST can manage multiple evidentiary types, making it more suitable for complex uncertain scenarios.

Characteristics:

- **Uncertainty Representation:** Provides a framework for representing and reasoning with incomplete evidence.
- **Conflict of Evidence:** Allows combining multiple sources of evidence using Dempster's rule of combination.
- **Decision-Making Ability:** Derives measures like belief, probability, and plausibility to facilitate decision-making processes.

The Murder Mystery Example:

Imagine there is a murder that has taken place in a room with 4 people present - Alice (A), Bob (B), Charlie (C), and Dana (D). The victim is Bob, who was stabbed in the back with a knife.

We know for certain that:

- 1) No one entered or left the room
- 2) Bob did not commit suicide

This leaves us with the following possible scenarios for who committed the murder:

- Alice ($\{A\}$) did it alone
- Charlie ($\{C\}$) did it alone
- Dana ($\{D\}$) did it alone
- Alice and Charlie ($\{A,C\}$) did it together
- Charlie and Dana ($\{C,D\}$) did it together
- Alice and Dana ($\{A,D\}$) did it together
- All three of them ($\{A,C,D\}$) did it together
- None of them ($\{\emptyset\}$) committed the murder (unlikely scenario).

Using Dempster-Shafer theory, we can assign belief masses (m) to each of these scenarios based on the evidence available. For example, if there is strong evidence implicating Alice, we may assign a high belief mass to the scenario $\{A\}$.

The key aspects are:

1) The belief masses assign our degree of belief/uncertainty to the different scenarios being true based on the evidence. The sum of all belief masses is 1.

2) Using Dempster's Rule of Combination, we can combine belief masses from different evidence sources to get updated/combined belief masses.

3) From the combined belief masses, we can calculate:

- **Belief in a Conclusion:** It represents the combined mass of evidence supporting a specific conclusion. For example, if there is strong evidence pointing towards A as the murderer, the belief in the conclusion "A committed the murder" would be high.
- **Plausibility of a Conclusion:** It represents the combined mass of evidence supporting a conclusion and its subsets. For instance, if there is evidence suggesting both A and C as potential murderers, the plausibility of the conclusion "A committed the murder" and "C committed the murder" would be elevated.

As more evidence comes in, we update the belief masses accordingly using Dempster's rule. The belief and plausibility in different scenarios guide our conclusion.

So DST allows us to effectively model uncertainty, combine evidence from multiple sources, and make decisions based on the degree of belief in different scenarios - which is powerful for AI reasoning under uncertainty.

Advantages:

As more information is added, uncertainty intervals decrease.

Can represent complex hierarchies in diagnostic scenarios.

Users have the freedom to interpret evidence based on their understanding.

Disadvantages:

Handling a large number of events/sources can lead to performance challenges.

Combining evidence requires careful modeling and calibration for accurate outcomes.

Interpretation of belief and plausibility values may introduce biases in decision-making processes.

Fuzzy Logic:

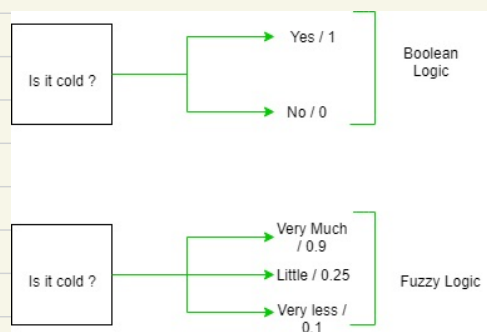
Fuzzy refers to situations where clarity or precision is lacking.

Fuzzy Logic is a form of multi-valued logic that allows for partial truths, not just true/false.

Based on the idea that many situations cannot be defined as completely true or false.

Provides a way to represent and reason with vagueness/uncertainty.

In the boolean system truth value, 1.0 represents the absolute truth value and 0.0 represents the absolute false value. But in the fuzzy system, there is no logic for the absolute truth and absolute false value. But in fuzzy logic, there is an intermediate value too present which is partially true and partially false.



Some of the core concept of fuzzy logic:

- **Membership Function:** Maps input values to a degree of membership between 0 and 1.

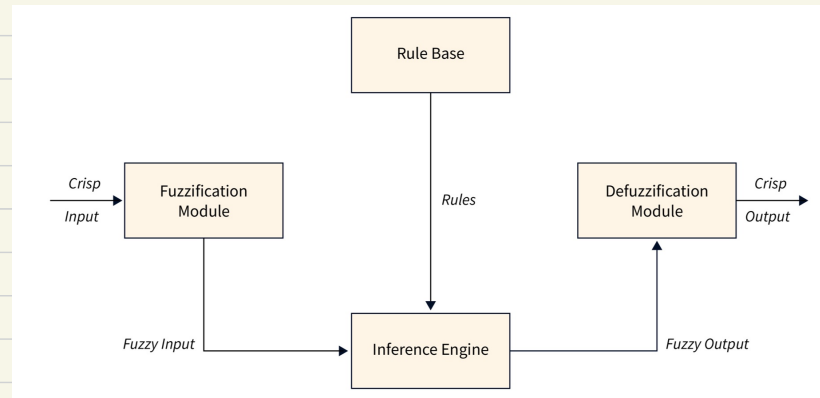
0 means no membership, 1 means full membership. It defines the fuzzy sets and degrees to which inputs belong.

- **Fuzzy Rules:** If-then statements expressing relationships between inputs and outputs. IF-THEN rules that capture relationships between inputs and outputs. It uses fuzzy values and variables, not just true/false.

Example: "IF temperature is HIGH and humidity is LOW, THEN AC is ON."

- **Fuzzy Sets:** Represent the output as a set of membership degrees for each possible value.

Fuzzy Logic Systems Architecture:



1. Input Variables: Define variables that system will process.
2. Fuzzification Interface: Converts real-world inputs into fuzzy sets using membership functions.
3. Fuzzy Rule Base: Collection of fuzzy if-then rules governing system behavior.
4. Inference Engine: Processes inputs according to rules, generating fuzzy output.
5. Defuzzification Interface: Translates fuzzy output into a crisp output for action.

Advantages of Fuzzy Logic:

- Works with imprecise, distorted, or noisy inputs.
- Easy construction and understandable.
- Based on set theory and simple reasoning.
- Efficient solution to complex problems, resembling human decision-making.
- Requires little memory due to compact algorithms.

Disadvantages of Fuzzy Logic:

- Lack of systematic approach, leading to ambiguity.
- Difficulty in mathematically proving characteristics.
- Accuracy compromised due to handling precise and imprecise data.

Applications:

- Aerospace: Altitude control of spacecraft.
- Automotive: Speed control, traffic control.
- Business: Decision support systems, personal evaluation.
- Chemical industry: pH control, chemical distillation.
- Artificial Intelligence: Natural language processing, expert systems.
- Modern control systems: Integration with neural networks for faster decision-making.

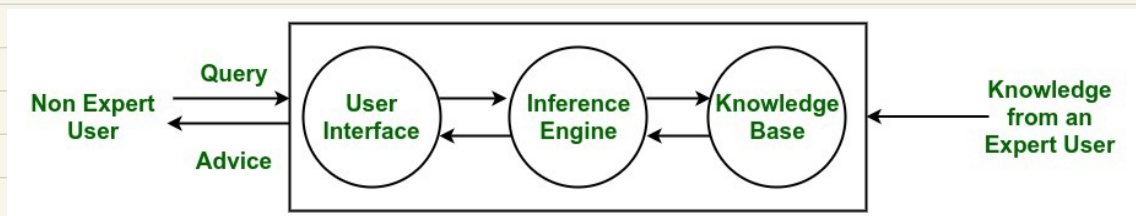
Expert Systems in AI:

- Software programs that mimic the decision-making ability of human experts in specific domains.
- Contain a knowledge base with information from experts.
- Use inference rules to draw conclusions and make recommendations, similar to human reasoning.

Purpose:

- To capture and preserve the knowledge and expertise of human experts.
- To provide advice, recommendations, and solutions to non-expert users.
- To make expert-level decisions consistently and efficiently.

Key Components:



1) Knowledge Base:

- Contains domain-specific knowledge from human experts.
- Includes facts, rules, procedures, and case data.

2) Inference Engine:

- Applies logical rules to known facts to derive new facts or conclusions.
- Uses techniques like forward chaining and backward chaining.

3) Knowledge Acquisition Module:

- Allows the system to acquire new knowledge from experts or data sources.
- Continuously expands and updates the knowledge base.

4) User Interface:

- Enables users to interact with the system and provide input.
- Presents recommendations, solutions, and explanations.

5) Explanation Module:

- Provides justifications and reasoning for the system's conclusions.
- Increases transparency and trust in the system.

Examples:

- MYCIN: Early expert system for diagnosing and treating bacterial infections.
- DENDRAL: System for analyzing molecular structures from spectrographic data.
- XCON: Configured computer systems based on user requirements.

Characteristics of Expert Systems:

- Permanence: Expertise remains available regardless of human availability.
- Knowledge Distribution: Shares expertise across users.
- Efficiency: Integrates knowledge from multiple experts.
- Cost Reduction: Reduces consulting costs, e.g., medical diagnosis.
- Utilizes knowledge base and inference engine.
- Solves complex problems through deduction.

Development Process:

- 1) Domain experts provide knowledge and expertise.
- 2) Knowledge engineers structure and encode the knowledge into the system.
- 3) End-users interact with the system for advice and solutions.

Advantages:

- Low accessibility cost.
- Fast response time.
- Emotion-free decision-making.
- Low error rate.
- Provides explanations for decisions.

Disadvantages:

- Lacks common sense and true intelligence.
- Domain-specific limitations.
- Manual updates required.
- Limited ability to explain decision logic.

Applications:

1) Medical diagnosis (internal medicine, blood diseases). 2) Software development project diagnosis. 3) Experiment planning (biology, chemistry). 4) Crop damage forecasting. 5) Manufacturing task scheduling. 6) Geologic structure assessment. 7) VLSI system design. 8) Education (teaching specialized tasks). 9) Legal assessments (civil cases, product liability).

Unit 4 → Procedural Knowledge and Declarative Knowledge:

Procedural Knowledge: Procedural knowledge, also known as imperative knowledge, is a type of knowledge that specifies how a particular task or problem can be accomplished. It emphasizes the "how-to" aspect of solving a problem. Procedural knowledge is typically represented as a set of rules or instructions that describe the steps or procedures to be followed.

Example:

```
var a = [1, 2, 3, 4, 5];  
var b = [];  
for (var i = 0; i < a.length; i++) {  
  b.push(a[i]);  
}  
console.log(b);
```

Output: [1, 2, 3, 4, 5]

In this example, the procedural knowledge is represented by the step-by-step instructions in the loop, which iterates over the elements of the array *a* and pushes each element into the new array *b*.

Declarative Knowledge: Declarative knowledge, also known as functional knowledge, is a type of knowledge that represents facts, concepts, and relationships about the world. It emphasizes the "what" aspect of knowledge rather than the "how." Declarative knowledge is typically represented in a more structured format, such as rules, facts, or logical statements.

Example:

```
var a = [1, 2, 3, 4, 5];  
var b = a.map(function(number) {  
  return number * 1;  
});  
console.log(b);
```

Output: [1, 2, 3, 4, 5]

In this example, the declarative knowledge is represented by the use of the *map* function, which operates on each element of the array *a* and applies the provided function (*number * 1*, which simply returns the number itself). The declarative knowledge here is the concept of mapping a function over an array to create a new array.

Both examples produce the same output, but they differ in their approach and the type of knowledge used. Procedural knowledge focuses on the step-by-step instructions, while declarative knowledge focuses on describing the facts and relationships.

In AI systems, both types of knowledge are important and can be used in different contexts. Procedural knowledge is often used for tasks that require specific sequences of actions, while declarative knowledge is useful for representing and reasoning about general facts and concepts.

Procedural Knowledge	VS Declarative Knowledge
Procedural knowledge provides the knowledge of how a particular task can be accomplished.	Declarative knowledge provides the basic knowledge about something.
Procedural knowledge is also termed as imperative knowledge.	Declarative knowledge is also termed as functional knowledge
Procedural knowledge revolves around the "How" of the concept.	Declarative knowledge revolves around the "What" of the concept.
Procedural knowledge is difficult to communicate.	Declarative knowledge is easily communicable.
Procedural knowledge is process-oriented.	Declarative knowledge is data-oriented.
Validation is not very easy in procedural knowledge.	Validation is quite easy in declarative knowledge.
Debugging is not very easy in procedural knowledge.	Debugging is quite easy in declarative knowledge.
Procedural knowledge is less commonly used.	Declarative knowledge is more general.
Procedural knowledge is represented by a set of rules.	Declarative knowledge is represented by production systems.
Procedural knowledge is obtained from actions, experiences, subjective insights, etc.	Declarative knowledge is obtained from principles, procedures, concepts, processes, etc.

Logic programming:

Logic programming is a programming paradigm where logical statements themselves are viewed as programs. The core idea is to represent knowledge using logical rules or assertions.

These rules take the form of Horn clauses, which are clauses with at most one positive literal. Examples are: P , $P \vee Q$, $P \rightarrow Q$

Horn clauses are used for two main reasons:

- 1) Their uniform representation allows writing simple and efficient interpreters.
- 2) The logic of Horn clauses is decidable, meaning there are algorithms to determine if a set of clauses is satisfiable or not.

In logic programming languages like Prolog, programs are actually sets of Horn clauses transformed as:

1. If a clause has no negative literals, leave it as is.
2. If it has negative literals, rewrite it as an implication combining the negatives in the antecedent and the positive in the consequent.
- 3.

For example, the Prolog clause $P(x) :- Q(x,y)$ is equivalent to the logical statement $\forall x \exists y Q(x,y) \rightarrow P(x)$.

This transformation causes clauses with disjunctions of literals (one positive) to become implications with:

- Antecedent as a conjunction of existentially quantified variables (from negative literals)
- Consequent with universally quantified variables (from positive literal)

The key difference between logical and Prolog representations is that Prolog has a fixed control strategy determining how to search for answers, while pure logical assertions only define the set of answers without specifying how to find them.

For example, consider the knowledge:

1. Logical Representation:

- $\forall x : \text{pet}(x) \wedge \text{small}(x) \rightarrow \text{apartmentpet}(x)$
- $\forall x : \text{cat}(x) \vee \text{dog}(x) \rightarrow \text{pet}(x)$
- $\forall x : \text{poodle}(x) \rightarrow \text{dog}(x) \wedge \text{small}(x) \wedge \text{poodle}(\text{fluffy})$

2. PROLOG Representation:

- $\text{apartmentpet}(x) \leftarrow \text{pet}(x), \text{small}(x)$
- $\text{pet}(x) \leftarrow \text{cat}(x)$
- $\text{pet}(x) \leftarrow \text{dog}(x)$
- $\text{dog}(x) \leftarrow \text{poodle}(x)$
- $\text{small}(x) \leftarrow \text{poodle}(x)$
- $\text{poodle}(\text{fluffy})$

Forward and Backward Reasoning

Forward Reasoning: Forward reasoning is a process in AI where we start with the initial data and facts, and then try to find all possible solutions or conclusions that can be derived from that initial information. It's a data-driven approach where we follow the direction from the given facts towards the conclusions.

In forward reasoning, the AI system is first provided with one or more constraints or facts. It then searches its knowledge base to find rules that match these given constraints. Any rule whose conditions are satisfied by the provided facts is triggered. The conclusions derived from these triggered rules generate new facts. These new facts are then added to the initial set of facts, and the process repeats with the AI continuing to apply more rules that match the incremented set of facts and constraints. This cycle continues until no new facts can be derived.

So in conclusion, forward reasoning follows a top-down approach, starting from the initial data and gradually building up towards the conclusions by chaining together rules from the knowledge base. It takes an opportunistic path, exploring all possible directions as new data arises.

Backward Reasoning: Backward reasoning works in the opposite direction compared to forward reasoning. Here, we start with a goal or a hypothesis, and then reason backwards to try to find the initial facts and rules that could support or derive that goal.

The AI system first selects a goal state that it wants to prove or establish. It then looks for rules in its knowledge base whose conclusions match this goal. For each such rule, it treats the rule's conditions as sub-goals that need to be satisfied for the main goal to be derived. The system then tries to find facts in its initial data that satisfy these sub-goals.

If the system can find a set of initial facts that satisfies all the sub-goals of a rule, then that rule's conclusion (the original goal) is considered proven based on those facts. However, if no set of initial facts can satisfy all the sub-goals simultaneously, then the goal is rejected.

Backward reasoning follows a conservative, goal-driven approach moving bottom-up from the desired conclusions towards the initial supporting facts and rules. It is also called decision-driven or hypothesis-driven reasoning.

Forward Reasoning	VS	Backward Reasoning
It is a data-driven task.		It is a goal driven task.
It begins with new data.		It begins with conclusions that are uncertain.
The objective is to find a conclusion that would follow.		The objective is to find the facts that support the conclusions.
It uses an opportunistic type of approach.		It uses a conservative type of approach.
It flows from incipient to the consequence.		It flows from consequence to the incipient.
Forward reasoning begins with the initial facts.		Backward reasoning begins with some goal (hypothesis).
Forward reasoning tests all the rules.		Backward reasons tests some rules.
Forward reasoning is a bottom-up approach.		Backward reasoning is a top-down approach.
Forward reasoning can produce an infinite number of conclusion.		Backward reasoning produces a finite number of conclusions.
In the forward reasoning, all the data is available.		In the backward reasoning, the data is acquired on demand.
Forward reasoning has a small number of initial states but a large number of conclusions.		Backward reasoning has a smaller number of goals and a larger number of rules.
In forward reasoning, the goal formation is difficult.		In backward reasoning, it is easy to form a goal.
Forward reasoning works in forward direction to find all the possible conclusions from facts.		Backward reasoning work in backward direction to find the facts that justify the goal.
Forward reason is suitable to answer the problems such as planning, control, monitoring, etc.		Backward reasoning is suitable for diagnosis like problems.

Matching

Matching is the process of comparing two or more structured representations like logic statements, networks etc. to find similarities and differences between them. It is a fundamental operation used across many AI applications like speech recognition, natural language processing, computer vision, expert systems etc.

There are different types of matching techniques:

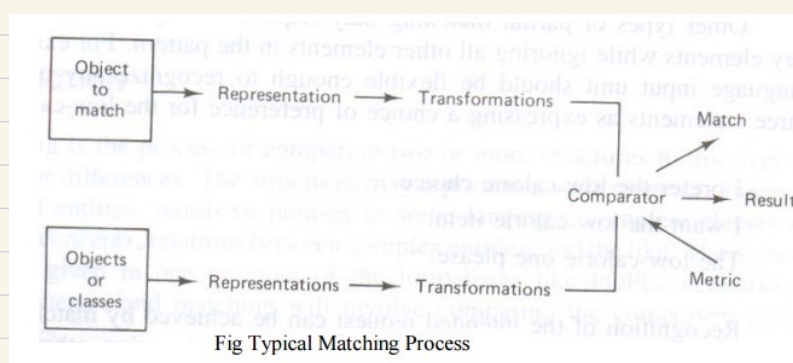
- 1) Exact Matching: Checks if two representations are precisely equal, with no transformations allowed.
- 2) Partial Matching: Allows transformations like variable binding or ignoring certain components to achieve a match. Aims to find the best possible match between representations. For example, recognizing "low-calorie" as the key intent across phrases like "I prefer the low-calorie choice", "I want the low-calorie item", etc.
- 3) Fuzzy Matching: Computes degrees of membership across multiple classes when boundaries are not clearly defined.

The matching process typically involves:

1. Transforming input representations to a common formalism if required
2. Comparing them component-wise using a similarity test
3. Combining the component similarities using an overall measure
4. Producing the match output (yes/no, bindings, annotations etc.)

Key factors that determine a matching algorithm are:

- The representation scheme
- The matching criteria (exact, partial, fuzzy)
- The similarity measure based on the criteria
- The required output format



Heuristic search techniques

In AI, many problems are too complex to be solved by traditional search algorithms within practical time and space limits. This leads to the use of heuristic search techniques that leverage heuristic functions to guide the search more efficiently.

Heuristic algorithms are not truly intelligent themselves, but they appear intelligent because they achieve better performance compared to blind search methods. They work by taking advantage of feedback from the data to direct the search path towards more promising areas of the solution space.

There are two main categories of search algorithms:

1. Uninformed/Brute-force search algorithms explore the entire search space systematically, checking all possible candidates to see if they satisfy the problem statement. Examples are breadth-first, depth-first, etc.
2. Informed search algorithms use heuristic functions specific to the problem domain to guide the search towards likely solution regions, reducing the amount of work needed.

A good heuristic function can make an informed search dramatically outperform uninformed search methods. For example, in the Traveling Salesperson Problem, the goal is to find a reasonably good solution quickly rather than the absolute best solution. Heuristics help predict which search paths are more promising and follow those, though not guaranteeing the optimal solution.

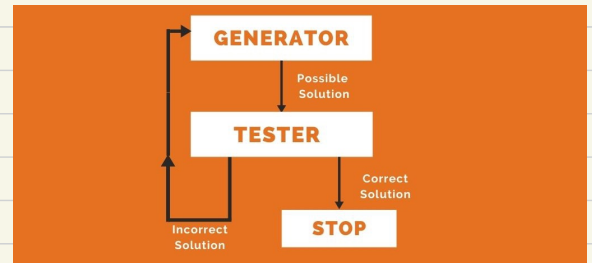
Such techniques allow finding satisfactory solutions within reasonable time and memory constraints. Some prominent informed search algorithms are:

- 1) Generate and Test - Generate candidate solutions and test if they satisfy the problem.
- 2) Best-first Search - Explore the most promising node first based on an evaluation function.
- 3) Greedy Search - Make the locally optimal choice at each step with the hope of finding a global optimum.
- 4) A* Search - An optimal best-first search that uses a heuristic to focus on paths likely to lead to the solution.

The key idea behind heuristic search is to use domain-specific knowledge encoded as heuristic functions to intelligently prune the search space and focus effort on fruitful regions, trading off completeness for efficiency when finding an optimal solution is impractical.

Generate and Test

Generate and Test is a heuristic search technique based on depth-first search with backtracking. It systematically generates candidate solutions and tests each one to see if it satisfies the problem's requirements. It guarantees finding a solution, if one exists, by exhaustively exploring the search space.



The algorithm works as follows:

1. Generate a possible solution (e.g. a state, path from start)
2. Test if this is an actual goal solution by checking against acceptable goal state(s)
3. If a solution is found, return it. Otherwise, go back to step 1 and generate a new candidate.

So it keeps generating candidates until a satisfactory solution emerges after testing them all systematically.

This technique is nicknamed the "British Museum" algorithm, likening it to randomly wandering around a museum looking for a specific exhibit.

To make it more efficient, a heuristic function can guide and prune the generate phase by ranking/prioritizing more promising candidates. This avoids wasted effort on unpromising paths.

For complex problems, generate and test alone may be inefficient. It can be combined with other techniques like constraint satisfaction to first reduce the search space before applying systematic generation and testing on the reduced space (like in the DENDRAL AI program).

For the generate phase to be effective, the candidate solution generators should have some key properties:

- 1) Completeness - able to generate all possible solutions to guarantee finding an answer
- 2) Non-redundancy - not generate duplicate solutions to avoid wasted work
- 3) Informedness - maintain knowledge about the search space, distances to goal etc. to prioritize generations

Some real-world analogies of generate and test include:

- Infinite monkeys generating Shakespeare's works through random typing
- DENDRAL program inferring molecular structures by generating candidates from spectrogram data

Hill Climbing

Hill Climbing is a local search optimization algorithm that tries to find the best solution to a problem by continually moving in the direction of increasing value or elevation. It starts from an initial state and keeps climbing by transitioning to neighboring states that have better objective function values, until it reaches a peak where no further improvement is possible among the neighbors.

The key aspects of the Hill Climbing algorithm are:

- 1) It is a variant of the "Generate and Test" method, where candidate neighbor states are generated and tested to see if they are better than the current state.
- 2) It takes a greedy approach by always moving to the best immediate neighbor state, without considering the overall optimal path.
- 3) There is no backtracking involved - it does not revisit previous states once it moves away from them.
- 4) It requires a good heuristic function to guide the climb towards promising areas of the search space.
- 5) The search space is visualized as a landscape, with peaks representing good solutions and valleys being poor ones. Hill Climbing aims to reach the highest peaks.

There are three main variants:

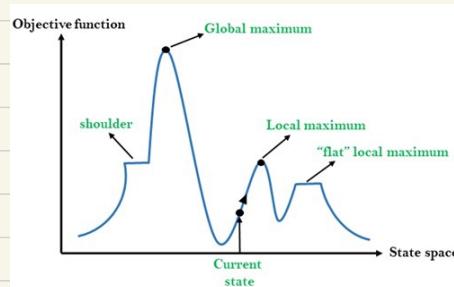
1. Simple Hill Climbing: Evaluates only one successor state and moves there if it is better than the current state.

Algorithm for Simple Hill Climbing:

- Step 1: Evaluate the initial state, if it is goal state then return success and Stop.
- Step 2: Loop Until a solution is found or there is no new operator left to apply.
- Step 3: Select and apply an operator to the current state.
- Step 4: Check new state:
 - 1) If it is goal state, then return success and quit.
 - 2) Else if it is better than the current state then assign new state as a current state.
 - 3) Else if not better than the current state, then return to step2.
- Step 5: Exit.

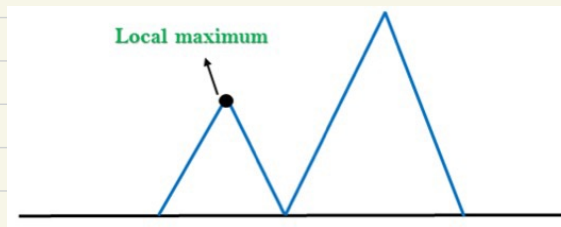
2. Steepest Ascent: Evaluates all successor states and moves to the best neighbor, even if not strictly better than the current state.

3. Stochastic: Randomly selects a successor state and decides whether to move there or examine another.

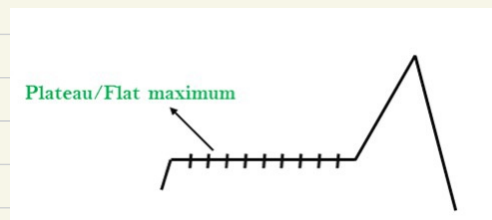


While climbing, Hill Climbing can get stuck at:

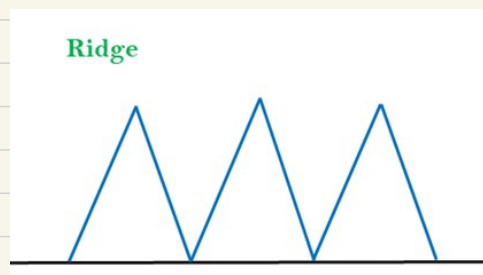
- Local Maxima: Peaks that are suboptimal compared to other higher peaks in the landscape.



- Plateaus: Flat regions where all neighbors have equal value, providing no sense of direction.



- Ridges: Elongated areas that are better than neighbors, but not peaks.



Potential solutions include backtracking, using random restarts, combining with other techniques, and altering step sizes.

The main strengths of Hill Climbing are its simplicity and efficiency when good heuristics are available. However, it is not complete and can get trapped in suboptimal regions. It works best for landscapes with small numbers of peaks.

Best First Search

Best First Search is an informed search algorithm that tries to find the optimal solution by exploring the most promising paths first. Unlike breadth-first and depth-first searches which explore blindly, Best First Search uses an evaluation function to estimate which neighboring nodes are most likely to lead to the goal, and expands those nodes first.

The key features are:

- 1) It falls under the category of heuristic or informed search techniques that leverage problem-specific knowledge.
- 2) It uses a priority queue to sort the unexplored paths based on an evaluation function value.
- 3) At each step, it removes and expands the most promising node/path from the priority queue.
- 4) The algorithm is a variation of breadth-first search, but instead of a regular queue, it uses a priority queue sorted by the evaluation function.

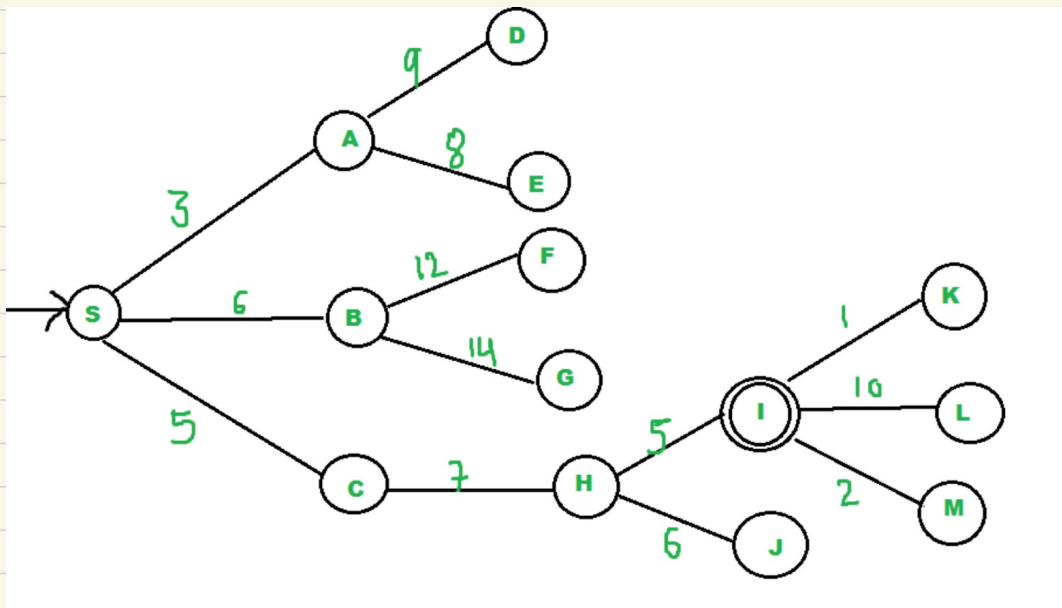
The algorithm works as follows:

- 1) Create an empty priority queue
- 2) Insert the start node into the queue
- 3) Until the queue is empty:
 - Remove the minimum evaluation score node from the queue
 - If it is the goal, return success
 - Else, loop through its unexplored neighbors:
 - Mark them as visited
 - Insert into the priority queue

So at each step, the most promising path based on the evaluation function is expanded, in an attempt to quickly reach the goal without exploring unpromising paths.

The time complexity is $O(n \log n)$ in the worst case of exploring all nodes, due to the $\log n$ cost of priority queue operations.

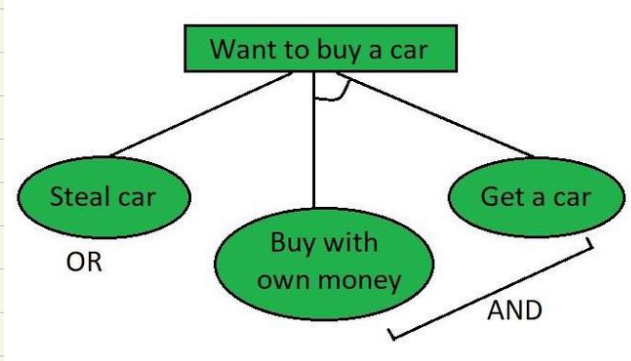
The performance crucially depends on the design of the evaluation/heuristic function that estimates costs and guides which paths to prioritize. A good heuristic can greatly reduce the effective branching factor.



As an example, consider finding the shortest path from S to I with given path costs: The queue initially contains S. Then it expands to {A, C, B}, sorted by costs, removing A first. Then {C, B, E, D}, removing C, and so on, ultimately reaching I via H due to the lowest cost.

Problem Reduction

In some AI problems, the overall problem can be broken down into a set of sub-problems. Solving each sub-problem separately and combining their solutions yields the solution to the original larger problem. Such decomposable problems are represented using AND-OR graphs or trees.



AND nodes represent sub-problems where all successor nodes (sub-problems) must be solved to solve the parent node. OR nodes represent alternative solution paths, where only one of the successor nodes needs to be solved.

The standard A* algorithm cannot efficiently handle AND-OR graphs because at AND nodes, the cost estimates depend not just on the current node's value, but also on the cumulative costs through the various successor paths that must be explored.

The AO* (AND-OR star) algorithm is a modification of A* designed to search AND-OR graphs effectively. Instead of maintaining separate OPEN and CLOSED lists, AO* uses a single graph structure G representing the explored search space so far.

Each node in G stores:

- 1) Pointers to immediate predecessors and successors
- 2) h' value - estimated cost from that node to the solution set

The algorithm works as follows:

- 1) Start with G containing just the initial state node $INIT$. Compute $h'(INIT)$.
- 2) Repeat until $INIT$ is solved or $h'(INIT)$ exceeds a futility threshold:
 - a) Trace marked arcs from $INIT$ to find an unexpanded node $NODE$
 - b) Generate $NODE$'s successors, adding them to G
 - c) Propagate new cost estimates up from $NODE$, updating h' values
 - d) Mark the currently best partial path out from each updated node
- 3) If $INIT$ was solved, the marked path from $INIT$ is the solution tree

The key features are:

- Exploring the currently best partial path incrementally
- Propagating new cost estimates up the graph after expanding nodes
- Marking the new best partial paths after Cost updates
- Ability to handle AND nodes requiring all successor costs

This allows AO^* to find the minimum cost solution in AND-OR graphs effectively, unlike A^* which can get stuck at AND nodes by underestimating costs.

However, AO^* has higher memory overhead from storing the entire explored graph G . Its time complexity also remains exponential in the worst case for solving intractable problems.

Constraint Satisfaction Problems

Constraint Satisfaction Problems (CSPs) are a category of AI problems where the goal is to find values for a set of variables that satisfy a given set of constraints or restrictions. They are widely used in AI applications involving resource allocation, planning, scheduling, and decision-making.

A CSP is defined by three main components:

- 1) Variables: These are the unknowns that need to be determined. For example, in a Sudoku puzzle, the variables represent the cells that need to be filled with numbers.
- 2) Domains: Each variable has a domain, which is the set of possible values that can be assigned to it. Domains can be finite (e.g., numbers 1-9 for Sudoku cells) or infinite.
- 3) Constraints: These are the rules or restrictions that govern how variables are related and what value combinations are valid or invalid. Constraints limit the values variables can take based on the values of other variables.

Some examples of constraints:

- Unary constraint on a single variable (e.g., X cannot take value 5)
- Binary constraint between two variables (e.g., $X \neq Y$)
- Higher-order constraints over multiple variables (e.g., $X+Y=10$)

The goal is to find an assignment of values to all variables that satisfies all the given constraints simultaneously.

Common CSP algorithms include:

- 1) Backtracking: A depth-first search that systematically assigns values to variables, undoing (backtracking) if a constraint violation occurs, until all variables are consistently assigned.
- 2) Forward Checking: An optimization over backtracking that removes future variable value choices that violate constraints after making each assignment.
- 3) Constraint Propagation: Using inference to reduce variable domains, propagating constraint effects to prune inconsistent values early on.

These techniques aim to efficiently explore the search space and satisfy all constraints by making legal value assignments to variables that do not conflict with each other.

CSPs provide a powerful general framework for modeling and solving a wide variety of real-world constraint-based problems like scheduling, resource planning, configuration, and design tasks.

The key features are:

- Modeling the problem variables, value choices, and constraints linking them
- Using smart search algorithms to explore the constrained solution space efficiently
- Propagating constraints to prune inconsistent possibilities early
- Finding one or all possible assignments satisfying all constraints

Means-Ends Analysis (MEA)

Means-Ends Analysis (MEA) is a problem-solving technique in AI that combines forward reasoning from the initial state and backward reasoning from the goal state. It aims to find a solution by reducing the differences between the current state and the desired goal state.

The core idea behind MEA is to:

- 1) Identify the key differences between the current state and goal state
- 2) Select operators (actions) that can reduce those differences
- 3) Apply those operators to generate new states closer to the goal
- 4) Recursively repeat this process on the new states until the goal is reached

MEA works by continuously evaluating the differences with the goal, and taking operators that make progress in reducing those differences, rather than exploring blindly. It interleaves forward and backward reasoning in a goal-directed manner.

The general algorithm is:

- 1) Compare current state to goal state
- 2) If no differences, return success
- 3) Else, select the most significant difference
- 4) Choose an operator to reduce that difference
- 5) If no such operator, signal failure
- 6) Otherwise, recursively apply MEA:
 - a) Find subgoals for operator's preconditions (backward chaining)
 - b) Try to achieve those subgoals from current state (forward chaining)
 - c) If both subproblems succeed, apply operator to reach closer state
 - d) Repeat from new closer state

This allows breaking down a problem into subgoals via operator applications, until all subgoals are achieved and the overall solution emerges by combining the sub-solutions.

MEA is a powerful technique that guides the problem-solving process more efficiently than blind search methods. It uses domain knowledge about operators and subgoal interactions to focus search efforts.

However, MEA can get stuck if no operators reduce remaining differences, or if the subgoal interactions are too complex. It works best for problems where the differences between states are easily identified and repairable via a sequence of operators.

The Means-Ends Analysis approach was first used in the General Problem Solver (GPS) program and has been applied to various domains like planning, robotics, theorem proving and games. It exemplifies the general idea of refining a problem representation until the solution becomes transparent.